

Símbolo	Tipo	Posición
5	L	00
10	L	00
'x'	V	99
15	L	01
20	L	01
'y'	V	98
25	L	04
30	L	04
1	C	97
35	L	09
40	L	09
't'	V	95
45	L	14
50	L	14
55	L	15
60	L	15
99	L	16

Fig. 12.29 Tabla simbólica correspondiente al programa 12.28.

programa, por lo que el contador de instrucciones es todavía 00). El comando `input` indica que la siguiente división léxica es una variable (únicamente una variable puede aparecer en un enunciado `input`). Dado que `input` corresponde directamente a un código de operación LMS, el compilador simplemente tiene que determinar la posición de `x` en el arreglo LMS. El símbolo `x` no se encuentra en la tabla simbólica.

Por lo tanto, en la tabla simbólica se le inserta como la representación ASCII de `x`, se le da el tipo `V`, y en el arreglo LMS se le asigna a la posición 99 (el almacenamiento de datos se inicia en 99 y se asigna hacia atrás). El código LMS puede ser generado a partir de este enunciado. El código de operación 10 (el código de operación de lectura LMS) se multiplica por 100, y se le añade la posición de `x` (como quedó determinada en la tabla simbólica), para completar la instrucción. La instrucción es entonces almacenada en el arreglo LMS en la posición 00. El contador de instrucciones se aumenta en 1, porque se produjo una sola instrucción LMS.

```
El enunciado
15 rem check y == x
```

se divide léxicamente después. El número de línea 15 es buscado en la tabla simbólica (y no se encuentra). El número de línea se inserta como del tipo `L` y se le asigna la siguiente posición en el arreglo, 01 (recuerde que los enunciados `rem` no producen código, por lo que no se aumenta el contador de instrucciones).

```
El enunciado
20 if y == x goto 60
```

se divide a continuación. El número de línea 20 se inserta en la tabla simbólica, se le da el tipo `L` con la siguiente posición 01 en el arreglo LMS. El comando `if` indica que se tendrá que hacer una evaluación de condición. La variable `y` no se encuentra en la tabla simbólica, por lo que se inserta y dándosele el tipo `V` y la posición LMS 98. A continuación, se generan instrucciones LMS para evaluar la condición. Dado que no existe un equivalente directo en LMS para `if/goto`, debe de ser simulado, ejecutando un cálculo utilizando `x` y `y` y bifurcación, basándose en el resultado. Si `y` es igual a `x`, el resultado de restar `x` de `y` debe ser cero, por lo que para simular el enunciado `if/goto` la instrucción *branch zero* puede ser utilizada

con el resultado del cálculo. El primer paso requiere que se cargue `y` (de la posición 98 LMS) al acumulador. Esto produce la instrucción 01 +2098. A continuación se resta `x` del acumulador. Esto produce la instrucción 02 +3199. El valor que aparece en el acumulador puede ser cero, positivo o negativo. Dado que el operador es `==`, deseamos hacer una *bifurcación cero*. Primero, se busca la tabla simbólica por la posición de bifurcación (en este caso 60), misma que no se encuentra. Por lo tanto, 60 se coloca en el arreglo `flags` en la posición 03, y se genera la instrucción 03 +4200 (no podemos añadir la posición de bifurcación, porque todavía no hemos asignado en el arreglo LMS una posición a la línea 60). El contador de instrucciones se incrementa a 04.

```
El compilador sigue con el enunciado
25 rem increment y
```

El número de línea 25 se inserta en la tabla simbólica como de tipo `L` y se le asigna la posición LMS 04. El contador de instrucciones no se incrementa.

```
Cuando se divide léxicamente el enunciado
30 let y = y + 1
```

el número de línea 30 se inserta en la tabla simbólica como del tipo `L` y se le asigna la posición LMS 04. El comando `let` indica que la línea es un enunciado de asignación. Primero, todos los símbolos en la línea se insertan en la tabla simbólica (si todavía no están ahí). El entero 1 se añade a la tabla simbólica como del tipo `C` y se le asigna la posición LMS 97. A continuación, el lado derecho de la asignación se convierte de notación infija a postfija. A continuación se evalúa la expresión postfija (`y 1 +`). El símbolo `y` se localiza en la tabla simbólica y se inserta en la pila su posición correspondiente en memoria. El símbolo 1 también es localizado en la tabla simbólica y se inserta dentro de la pila su posición correspondiente en memoria. Cuando se llega al operador `+`, el evaluador de postfijo extrae (`pop`) la pila hacia el operando derecho del operador y vuelve a extraer la pila nuevamente en el operando izquierdo del operador, y a continuación produce las instrucciones LMS

```
04 +2089 (cargar y)
05 +3097 (añadir 1)
```

El resultado de la expresión se almacena en una posición temporal en memoria (96) mediante la instrucción

```
06 +2196 (almacenar temporal)
```

y la posición temporal se inserta en la pila. Ahora que ha sido evaluada la expresión, el resultado debe ser almacenado en `y` (es decir la variable del lado izquierdo del signo igual). Por lo tanto, la posición temporal se carga al acumulador y el acumulador se almacena en `y`, mediante las instrucciones

```
07 +2096 (cargar temporal)
08 +2198 (almacenar y)
```

El lector notará de inmediato que las instrucciones LMS son aparentemente redundantes. Analizaremos este tema en breve.

```
Cuando se divide léxicamente el enunciado
35 rem add y to total
```

el número de línea 35 se inserta en la tabla simbólica como del tipo `L` y se le asigna la posición 09.

```
El enunciado
40 let t = t + y
```

es similar a la línea 30. La variable se inserta en la tabla simbólica como en el tipo `V` y se le asigna la posición LMS 95. Las instrucciones siguen la misma lógica y formato que en la línea 30, y son generadas las instrucciones 09 + 2095, 10 +3098, 11 +2194, 12 +2094, y 13 +2195. Note que el resultado

de `t +` y se asigna a la posición temporal 94 antes de asignarse a `t` (95). Otra vez, el lector notará que las instrucciones en las posiciones de memoria 11 y 12 aparecen como redundantes. Otra vez, en breve analizaremos lo anterior.

```
El enunciado
45 rem    loop y
```

es un comentario, por lo que la línea 45 se añade a la tabla simbólica como del tipo `L` y se le asigna la posición LMS 14.

```
El enunciado
50 goto 20
```

transfiere el control a la línea 20. El número de línea 50 se inserta en la tabla simbólica como del tipo `L` y se le asigna a la posición LMS 14. En LMS el equivalente de `goto` es la instrucción *unconditional branch* (40) que transfiere control a una posición LMS específica. El compilador busca la línea 20 en la tabla simbólica y encuentra que corresponde a la posición LMS 01. El código de operación (40) se multiplica por 100 y la posición 01 se añade a esta cifra, para producir la instrucción 14 +4001.

```
El enunciado
55 rem    output result
```

es un comentario, por lo que la línea 55 se inserta a la tabla simbólica como del tipo `L` y se le asigna la posición LMS 15.

```
El enunciado
60 print t
```

es un enunciado de salida. El número de línea 60 se inserta en la tabla simbólica como del tipo `L` y se le asigna la posición LMS 15. El equivalente de `print` en código de operación LMS es 11 (*write*). La posición de `t` se determina a partir de la tabla simbólica y se añade al resultado de multiplicar el código de operación por 100.

```
El enunciado
99 end
```

es la línea final del programa. El número de línea 99 se almacena en la tabla simbólica como del tipo `L` y se le asigna la posición LMS 16. El comando `end` produce la instrucción LMS +4300 (en LMS 43 es *halt*) que se escribe como instrucción final en el arreglo de memoria LMS.

Esto completa la primera pasada del compilador. Ahora veamos la segunda pasada. Valores distintos a -1 son buscados dentro del arreglo `flags`. La posición 03 contiene 60, por lo que el compilador sabe que la instrucción 03 está incompleta. El compilador completa la instrucción buscando en la tabla simbólica el 60, determinando su posición y añadiendo la posición a la instrucción incompleta. En este caso, la búsqueda determina que la línea 60 corresponde a la posición LMS 15, por lo que en remplazo de 03 +4200 se produce la instrucción completa 03 +4215. Ahora el programa Simple ha quedado ya compilado con éxito.

Para construir el compilador tendrá que llevar a cabo cada una de las siguientes tareas.

- a) Modificar el programa simulador Simpletron, que usted escribió en el ejercicio 7.19, para que tome su entrada a partir de un archivo especificado por el usuario (vea el capítulo 11). También, el simulador deberá extraer sus resultados a un archivo de disco, con el mismo formato que la salida a pantalla.
- b) Modifique el algoritmo de evaluación infijo a postfijo del ejercicio 12.12, para que pueda procesar operandos enteros multidigitales, y operandos de nombres de variables de una sola letra. *Sugerencia:* puede ser utilizada la función estándar de biblioteca `strtok` para localizar dentro de una expresión cada constante y variable, y las constantes pueden ser convertidas de

cadenas a enteros utilizando la función estándar de biblioteca `atoi`. (Nota: deberá ser modificada la representación de datos de la expresión postfija, para que acepte nombres de variables y constantes enteras).

- c) Modifique el algoritmo de evaluación postfijo para que pueda procesar operandos enteros de más de un dígito y operandos de nombre de variable. También, el algoritmo deberá ahora poner en funcionamiento el “gancho” discutido anteriormente, de tal forma que sean producidas las instrucciones LMS en vez de que la expresión se evalúe directamente. Sugerencia: la función estándar de biblioteca `strtok` puede ser utilizada para localizar dentro de una expresión cada una de las constantes y variables, y las constantes pueden ser convertidas de cadenas a enteros utilizando la función estándar de biblioteca `atoi`. (Nota: deberá ser modificada la representación de datos de una expresión postfija, para que acepte nombres de variables y constantes enteras.)
- d) Construya el compilador. Incorpore las partes (b) y (c) para evaluar las expresiones en los enunciados `let`. Su programa deberá contener una función que ejecute la primera pasada del compilador, y una función que ejecute la segunda pasada del compilador. Para llevar a cabo sus tareas ambas funciones pueden llamar a otras funciones.

12.28 (Cómo optimizar el compilador Simple). Cuando se compila un programa y se convierte a LMS, se generan un conjunto de instrucciones. A menudo ciertas combinaciones de instrucciones se repiten, normalmente en grupos de tres, llamadas *producciones*. Una producción está formada normalmente de tres instrucciones, como *load*, *add*, y *store*. Por ejemplo, en la figura 12.30 se ilustran cinco de las instrucciones LMS, que fueron producidas en la compilación del programa de la figura 12.28. Las primeras tres instrucciones son la producción que añade 1 a `y`. Note que las instrucciones 06 y 07 almacenan el valor en el acumulador en la posición temporal 96. Y a continuación vuelven a cargar el valor en el acumulador, de tal forma que la instrucción 08 puede almacenar el valor en la posición 98. A menudo una producción es seguida por una instrucción de carga para la misma posición que fue almacenada inmediatamente antes. Este código puede ser *optimizado* eliminando la instrucción de almacenar y las instrucciones de cargar subsecuentes, que operan en la misma posición de memoria. Esta optimización permitiría que el programa Simpletron se ejecutara más aprisa, porque en esta versión existirían menos instrucciones. En la figura 12.31 se ilustra el LMS optimizado, correspondiente al programa de la figura 12.28. Note que en el código optimizado hay cuatro instrucciones menos —un ahorro de espacio en memoria del 25%.

Modifique el compilador para que pueda dar la opción de optimizar el código que produce en lenguaje de máquina Simpletron. Compare manualmente el código no optimizado con el código optimizado, y calcule el porcentaje de reducción.

12.29 (Modificaciones al compilador Simple). Lleve a cabo las siguientes modificaciones al compilador Simple. Algunas de estas modificaciones también pudieran requerir modificaciones al programa simulador Simpletron escrito en el ejercicio 7.19.

- a) Haga posible que en los enunciados `let` se pueda utilizar el operador de módulo (%). El lenguaje de máquina Simpletron deberá ser modificado para incluir la instrucción de módulo.
- b) Haga posible la exponenciación en un enunciado `let` utilizando el `^` como operador de exponenciación. El lenguaje de máquina Simpletron debe ser modificado para incluir la instrucción de exponenciación.

04	+2098	(load)
05	+3097	(add)
06	+2196	(store)
07	+2096	(load)
08	+2198	(store)

Fig. 12.30 Código sin optimizar del programa de la figura 12.28.

Programa simple	Localización e instrucción LMS		Descripción
5 rem sum 1 to x	ninguna		rem es ignorado
10 input x	00	+1099	leer x a la posición 99
15 rem check y == x	ninguna		rem es ignorado
20 if y == x goto 60	01	+2098	cargar y (98) a acumulador
	02	+3199	substraer x (99) del acumulador
	03	+4211	bifurcar cero a posición sin resolver
25 rem increment y	ninguna		rem es ignorado
30 let y = y + 1	04	+2098	cargar y al acumulador
	05	+3097	añadir 1 (97) al acumulador
	06	+2198	almacenar acumulador en y (98)
35 rem add y to total	ninguna		rem es ignorado
40 let t = t + y	07	+2096	cargar t de posición (96)
	08	+3098	añadir y (98) a acumulador
	09	+2196	almacenar acumulador en t (96)
45 rem loop y	ninguna		rem es ignordo
50 goto 20	10	+4001	bifurcarse a la posición 01
55 rem output result	ninguna		rem es ignorado
60 print t	11	+1195	extraer t (96) a pantalla
99 end	12	+4300	terminar la ejecución.

Fig. 12.31 Código optimizado para el programa de la figura 12.28.

- c) Haga posible que en los enunciados Simple el compilador reconozca letras mayúsculas y minúsculas (por ejemplo 'A' sea equivalente a 'a'). No son requeridas modificaciones al simulador Simpletron.
- d) Haga posible que los enunciados input puedan leer valores para variables múltiples, como input x, y. No se requieren modificaciones al simulador Simpletron.
- e) Haga posible que el compilador extraiga varios valores en un enunciado único print, como print a, b, c. No se requieren modificaciones al simulador Simpletron.
- f) Añada capacidades de verificación de sintaxis al compilador, de tal forma que cuando en un programa Simple se encuentren errores de sintaxis se emitan mensajes de error. No se requieren modificaciones al simulador Simpletron.
- g) Haga posible el uso de arreglos de enteros. No se requieren modificaciones al simulador Simpletron.
- h) Permitir subrutinas especificadas por los comandos Simple gosub y return. El comando gosub pasa el control del programa a una subrutina y el comando return pasa el control de regreso al enunciado existente después del gosub. Esto es similar a una llamada de función en C. La misma subrutina puede ser llamada a partir de muchos gosub distribuidos a lo largo de un programa. No se requiere de modificaciones al simulador Simpletron.
- i) Haga posible estructuras de repetición de la forma

for x = 2 to 10 step 2
 enunciados Simple
next

Este enunciado for cicla desde 2 hasta 10 con un incremento de 2. La línea next marca el fin del cuerpo de la línea for. No se requieren de modificaciones al simulador Simpletron.

j) Haga posible estructuras de repetición de la forma
 for x = 2 to 10
 enunciados Simple
next

Este enunciado for cicla desde 2 hasta 10 con un incremento preestablecido de 1. No se requieren de modificaciones al simulador Simpletron.

k) Haga posible que el compilador procese entradas y salidas de cadenas. Esto requiere la modificación del simulador Simpletron para procesar y almacenar valores de cadenas. *Sugerencia:* cada palabra Simpletron puede ser dividida en dos grupos, cada uno de los grupos conteniendo un entero de dos dígitos. Cada entero de dos dígitos representa el equivalente decimal ASCII de un carácter. Añada una instrucción en lenguaje máquina que imprimirá una cadena empezando en una posición de memoria Simpletron. La primera mitad de la palabra en dicha posición es una cuenta del número de caracteres dentro de la cadena (es decir la longitud de la cadena). Cada media palabra sucesiva contiene un carácter ASCII expresado como dos dígitos decimales. La instrucción en lenguaje máquina verifica la longitud e imprime la cadena traduciendo cada número de dos dígitos en su carácter equivalente.

l) Haga posible que además de enteros el compilador procese valores de punto flotante. También deberá ser modificado el simulador Simpletron para procesar valores de punto flotante.

12.30 (Un intérprete Simple). Un intérprete es un programa que lee un enunciado de un programa en un lenguaje de alto nivel, determina la operación a ejecutarse por dicho enunciado, y de inmediato procede a ejecutar la operación. El programa no se convierte primero a lenguaje máquina. Los intérpretes ejecutan lentamente, porque cada enunciado que se encuentra en el programa deberá primero ser descifrado. Si los enunciados están incluidos en un ciclo, cada vez que se encuentren dentro del ciclo, los enunciados serán descifrados. Las versiones primeras del lenguaje de programación BASIC eran puestas en operación por medio de intérpretes.

Escriba un intérprete para el lenguaje Simple analizado en el ejercicio 12.26. El programa deberá utilizar el convertidor de infijo a postfijo desarrollado en el ejercicio 12.12 y el evaluador postfijo desarrollado en el ejercicio 12.13 para evaluar las expresiones en un enunciado let. En este programa deberán ser impuestas las mismas restricciones puestas en práctica en el lenguaje Simple del ejercicio 12.26. Pruebe el intérprete con los programas Simple escritos en el ejercicio 12.26. Compare los resultados de ejecutar estos programas en el intérprete con los resultados de compilar programas Simple y ejecutarlos en el simulador Simpletron que se construyó en el ejercicio 7.19.

13

El preprocesador

Objetivos

- Ser capaz de utilizar **#include** para el desarrollo de programas extensos.
- Ser capaz de utilizar **#define** para crear macros y macros con argumentos.
- Comprender la compilación condicional.
- Ser capaz de mostrar mensajes de error durante la compilación condicional.
- Ser capaz de usar afirmaciones para probar si los valores de expresiones son correctos.

Sostén lo bueno; defínelo bien.

Alfred, Lord Tennyson

Le he encontrado un argumento; pero no estoy obligado de encontrarle una comprensión.

Samuel Johnson

Un buen símbolo es el mejor argumento, y es un misionero para persuadir a miles.

Ralph Waldo Emerson

Las condiciones son básicamente sanas.

Herbert Hoover (Diciembre 1929)

Al partidario, cuando está inmerso en una discusión, no le importan los derechos de la pregunta, sino que solo está ansioso de convencer a sus oyentes de sus propias afirmaciones.

Platón

Sinopsis

- 13.1 Introducción
- 13.2 La directiva de preprocesador #include
- 13.3 La directiva de preprocesador #define: constantes simbólicas
- 13.4 La directiva de preprocesador #define: macros
- 13.5 Compilación condicional
- 13.6 Las directivas de preprocesador #error y #pragma
- 13.7 Los operadores # y ##
- 13.8 Número de línea
- 13.9 Constantes simbólicas predefinidas
- 13.10 Asertos

Resumen • Terminología • Errores comunes de programación • Prácticas sanas de programación • Sugerencias de rendimiento • Ejercicios de autoevaluación • Respuestas a los ejercicios de autoevaluación • Ejercicios.

13.1 Introducción

Este capítulo presenta el *preprocesador C*. El preprocesamiento ocurre antes de que se compile un programa. Algunas acciones posibles son: inclusión de otros archivos en el archivo bajo compilación, definición de *constantes simbólicas* y de *macros*, *compilación condicional* de código de programa, y *ejecución condicional de directivas* de preprocesador. Todas las directivas de preprocesador empiezan con el signo #, y antes de una directiva de preprocesador en una línea sólo pueden aparecer espacios en blanco.

13.2 La directiva de preprocesador #include

La *directiva de preprocesador #include* ha sido utilizada a todo lo largo de este texto. La directiva **#include** hace que en el lugar de la directiva se incluya una copia del archivo especificado. Las dos formas de la directiva **#include** son:

```
#include <filename>
#include "filename"
```

La diferencia entre estas formas estriba en la localización donde el preprocesador buscará el archivo a incluirse. Si el nombre del archivo está encerrado entre comillas, el preprocesador buscará el archivo a incluirse en el mismo directorio que el del archivo que se está compilando. Este método se utiliza normalmente para incluir archivos de cabecera definidos por el programador. Si el nombre del archivo esta encerrado en corchetes angulares (< y >) —que se utilizan para los *archivos de cabecera estándar de biblioteca*— la búsqueda se llevará a cabo de forma independiente a la puesta en práctica, por lo regular a través de directorios predesignados.

La directiva **#include** se utiliza por lo general para incluir archivos de cabecera de biblioteca estándar, como **stdio.h** y **stdlib.h** (vea la figura 5.6). La directriz **#include** también se utiliza con programas formados de varios archivos fuente, que deben ser compilados juntos. A menudo es creado e incluido en el archivo un *archivo de cabecera* conteniendo declaraciones comunes a los varios archivos de un programa. Ejemplos de declaraciones como éstas son declaraciones de estructuras y de uniones, en numeraciones y prototipos de función.

En UNIX, los archivos de programa se compilan utilizando el *comando CC*. Por ejemplo, para compilar y enlazar **main.c** con **square.c** escriba el comando.

```
cc main.c square.c
```

en el indicador de UNIX. Esto produce el archivo ejecutable **a.out**. Para mayor información sobre la compilación, el enlace y la ejecución de programas, vea los manuales de referencia correspondientes a su compilador.

13.3 La directiva de preprocesador #define: constantes simbólicas

La *directiva #define* crea *constantes simbólicas* —constantes representadas como símbolos— y *macros*— operaciones definidas como símbolos. El formato de la directiva **#define** es

```
#define identifier replacement-text
```

Cuando en un archivo aparece esta línea, todas las subsecuentes apariciones de *identifier* serán de forma automática remplazadas por *replacement-text*, antes de la compilación del programa. Por ejemplo,

```
#define PI 3.14159
```

remplaza todas las ocurrencias subsiguientes de la constante simbólica **PI** por la constante numérica **3.14159**. Las constantes simbólicas permiten al programador ponerle un nombre a una constante y utilizarlo a todo lo largo del programa. Si durante el programa la constante requiere de modificación, puede ser modificada una vez en la directiva **#define** y cuando sea el programa recompilado, todas las ocurrencias de la constante dentro del programa de manera automática serán modificadas. Nota: *todo lo que exista a la derecha del nombre de la constante simbólica, remplaza a la constante simbólica*. Por ejemplo, **#define PI = 3.14159** hace que el preprocesador remplace todas las ocurrencias de **PI** con **= 3.14159**. Esto es causa de muchos errores sutiles de lógica y de sintaxis. Es también un error redefinir una constante simbólica con un nuevo valor.

Práctica sana de programación 13.1

Utilizar nombres significativos para las constantes simbólicas, ayuda a que los programas resulten más autodocumentados.

13.4 La directiva de preprocesador #define: macros

Una *macro* es una operación definida en una directiva de preprocesador **#define**. Como en el caso de las constantes simbólicas, antes de compilarse el programa, el *macro identifier* se remplaza en el mismo por el *replacement-text*. Las macros pueden ser definidas con o sin *argumentos*. Una macro sin argumentos se procesa como si fuera una constante simbólica. En una macro con argumentos, los argumentos se sustituyen en el texto de remplazo, y a continuación la macro se *expande*, es decir, en el programa el texto de remplazo remplaza al identificador y a la lista de argumentos.

Veamos la siguiente definición de macro con un argumento, correspondiente al área de un círculo:

```
#define CIRCLE_AREA(x) PI * (x) * (x)
```

Siempre que en el archivo aparezca **CIRCLE_AREA(x)**, el valor de **x** será sustituido por **x** en el texto de remplazo, la constante simbólica **PI** será remplazada por su valor (previamente definido), y en el programa la macro se expandirá. Por ejemplo, el enunciado

```
area = CIRCLE_AREA(4);
```

se expande a

```
area = 3.14159 * (4) * (4);
```

Dado que la expresión está sólo formada de constantes, al momento de compilarse se evalúa el valor de la expresión y se asigna a la variable **area**. Cuando el argumento del macro es una expresión, los paréntesis alrededor de cada **x** en el texto de remplazo obligan al orden apropiado de evaluación. Por ejemplo, el enunciado

```
area = CIRCLE_AREA(c + 2);
```

se expande a

```
area = 3.14159 * (c + 2) * (c + 2);
```

que se evalúa de forma correcta, porque los paréntesis obligan al orden correcto de evaluación. Si se omiten los paréntesis, la expansión del macro es

```
area = 3.14159 * c + 2 * c + 2;
```

lo que se evalúa de forma incorrecta como

```
area = (3.14159 * c) + (2 * c) + 2;
```

en razón de las reglas de precedencia de operadores.

Error común de programación 13.1

Olvidar encerrar los argumentos de macro en paréntesis en el texto de remplazo.

La macro **CIRCLE_AREA** podría ser definida como una función. La función **circleArea**

```
double circleArea(double x)
{
    return 3.14159 * x * x;
}
```

lleva a cabo el mismo cálculo que la macro **CIRCLE_AREA**, pero con la función **circleArea** se asocia la sobrecarga de una llamada de función. Las ventajas de la macro **CIRCLE_AREA** es que las macros insertan código en el programa directamente evitando sobrecarga de función y conservándose el programa legible, porque el cálculo **CIRCLE_AREA** es definido por separado y nombrado de forma significativa. Una desventaja estriba en que su argumento debe evaluarse dos veces.

Sugerencia de rendimiento 13.1

A veces las macros pueden ser utilizadas para sustituir una llamada de función por código en línea, previo al tiempo de ejecución. Esto elimina la sobrecarga de una llamada de función.

La siguiente es una definición de macro con dos argumentos, correspondiente al área de un rectángulo:

```
#define RECTANGLE_AREA(x, y) (x) * (y)
```

Siempre que en el programa aparezca **RECTANGLE_AREA(x, y)**, los valores de **x** y **y** serán sustituidos por el texto de remplazo de la macro, y la macro será expandida en lugar del nombre de la macro. Por ejemplo, el enunciado

```
rectArea = RECTANGLE_AREA(a + 4, b + 7);
```

se expande a

```
rectArea = (a + 4) * (b + 7);
```

El valor de la expresión es evaluado y asignado a la variable **rectArea**.

El texto de remplazo para una macro o una constante simbólica es por lo regular cualquier texto sobre la línea, después del identificador en la directriz **#define**. Si el texto de remplazo de una macro o de una constante simbólica es más largo que el resto de la línea, deberá colocarse una diagonal invertida (****) al final de la misma, indicando que el texto de remplazo continúa sobre la siguiente línea.

Las constantes simbólicas y las macros pueden ser descartadas utilizando la *directiva de preprocesador* **#undef**. La directiva **#undef** “elimina” la definición de una constante simbólica o de un nombre de macro. El *alcance* de una constante simbólica o de una macro cubre desde su definición hasta que mediante **#undef** queda eliminada su definición, o si no, hasta el final del archivo. Una vez eliminada la definición, puede volverse a definir un nombre utilizando **#define**.

Las funciones en la biblioteca estándar son definidas algunas veces como macros basadas en otras funciones de biblioteca. Una macro, comúnmente definida en el archivo de cabecera **stdio.h**, es

```
#define getchar() getc(stdin)
```

La definición de macro de **getchar** utiliza la función **getc** para obtener un carácter a partir del flujo de entrada estándar. A menudo la función **putchar** en el encabezado **stdio.h**, y las funciones de manejo de caracteres del encabezado **ctype.h**, también son puestas en operación como macros. Note que las expresiones con efectos colaterales (es decir, donde los valores de las variables se modifican) no deberían ser pasadas a una macro porque los argumentos de macro pudieran ser evaluados más de una vez.

13.5 Compilación condicional

La *compilación condicional* le permite al programador controlar la ejecución de las directivas de preprocesador, y la compilación del código del programa. Cada una de las directivas de preprocesador evalúa una expresión entera constante. Las expresiones de cambio de tipo, las expresiones **sizeof**, y las constantes de enumeración no pueden ser evaluadas en directivas de preprocesador.

El constructor de preprocesador condicional es muy parecido a la estructura de selección **if**. Veamos el siguiente código de preprocesador:

```
#if !defined(NULL)
#define NULL 0
#endif
```

Estas directivas determinan si **NULL** ha sido definido. La expresión **defined(NULL)** se evalúa a 1, si **NULL** está definido; y a 0 de lo contrario. Si el resultado es 0, **!defined(NULL)** se evalúa a 1, y **NULL** queda definido. De lo contrario, la directiva **#define** es pasada por alto. Cada constructor **#if** termina con **#endif**. Las directivas **#ifdef** y **#ifndef** son abreviaturas correspondientes a **#if defined(nombre)** y **#if !defined(nombre)**. Un constructor de preprocesador condicional de varias partes puede ser probado utilizando las directivas **#elif** (el equivalente de **else if** en una estructura **if**) y **#else** (el equivalente de **else** en una estructura **if**).

Durante el desarrollo del programa, a menudo los programadores encuentran útil "anotar" grandes porciones de código para evitar que sean compilados. Si el código contiene comentarios, para esta tarea no es posible utilizar **/*** y ***/**. En vez de ello, el programador puede recurrir al siguiente constructor de preprocesador

```
#if 0
    code prevented from compiling
#endif
```

Para permitir que el código sea compilado, en el constructor anterior el 0 es remplazado por 1.

La compilación condicional se utiliza por lo común como ayuda de depuración. Muchas instalaciones en C proporcionan *depuradores*. Sin embargo, con frecuencia los depuradores son difíciles de utilizar y de comprender, por lo que son rara vez utilizados por los estudiantes de un primer curso de programación. En vez de ello, se usan enunciados **printf** para imprimir valores de variables y para confirmar el flujo de control. Estos enunciados **printf** pueden ser encerrados en directivas de preprocesador condicionales, de tal forma que los enunciados sólo sean compilados mientras no se haya terminado el proceso de depuración. Por ejemplo

```
#ifdel DEBUG
    printf("Variable x = %d\n", x);
#endif
```

hace que se compile en el programa un enunciado **printf** si la constante simbólica **DEBUG** ha sido definida (**#define DEBUG**) antes de la directiva **#ifdef DEBUG**. Cuando se haya terminado la depuración, se elimina del archivo fuente la directiva **#define**, y durante la compilación los enunciados **printf**, insertos para fines de depuración, serán ignorados. En programas más extensos pudiera ser deseable definir varias constantes simbólicas distintas, que controlen la compilación condicional en secciones separadas del archivo fuente.

Error común de programación 13.2

Insertar enunciados printf compilados condicionalmente para fines de depuración, en posiciones donde en ese momento C espera un enunciado. En este caso, el enunciado compilado condicional debería haberse encerrado en un enunciado compuesto. Entonces, cuando el programa se compile con enunciados de depuración, el flujo del control del programa no será alterado.

13.6 Las directivas de preprocesador #error y #pragma

La directiva **#error**

#error tokens

imprime un mensaje, que depende de la puesta en práctica, incluyendo las *divisiones o componentes léxicas* especificadas en la directriz. Las divisiones léxicas son secuencias de caracteres separadas por espacios. Por ejemplo,

#error 1 - Out of range error

contiene 6 divisiones léxicas. En Borland C++ para PC, por ejemplo, cuando se procesa la directiva **#error**, se muestran las componentes o divisiones léxicas en la directiva como un mensaje de error, el preproceso se detiene, y el programa no se compila.

La directiva **#pragma**

#pragma tokens

genera una acción definida por la puesta en práctica. Un pragma no reconocido por la puesta en práctica será ignorado. Borland C++, por ejemplo, reconoce varios pragmas que le permiten al programador aprovechar completamente la puesta en marcha de Borland C++. Para mayor información sobre **#error** y **pragma**, vea la documentación en su instalación de C.

13.7 Los operadores # y

Los operadores de preprocesador **#** y **##** están disponibles sólo en ANSI C. El operador **#** hace que una división léxica de texto de remplazo se convierta en una cadena encerrada entre comillas. Considere la siguiente definición de macro:

```
#define HELLO(x) printf("Hello, " #x "\n");
```

Cuando **HELLO(John)** aparece en un archivo de programa, se expande a

```
printf("Hello, " "John" "\n");
```

La cadena "John" remplacea **#x** en el texto de remplazo. Las cadenas separadas por un espacio en blanco son concatenadas durante el preprocesamiento, por lo que el enunciado arriba citado es equivalente a

```
printf("Hello, John\n");
```

Note que el operador **#** deberá ser utilizado en una macro con argumentos, porque el operando de **#** se refiere a un argumento de la macro.

El operador **##** concatena dos componentes léxicos. Considere la siguiente definición de macro:

```
#define TOKENCONCAT(x, y) x ## y
```

Cuando en el programa aparece **TOKENCONCAT**, sus argumentos son concatenados y utilizados para remplazar la macro. Por ejemplo, en el programa **TOKENCONCAT(O, K)** es remplazado por **OK**. El operador **##** debe tener dos operandos.

13.8 Números de línea

La directiva de preprocesador **#line** hace que las líneas de código fuente subsecuentes se renumeran, iniciándose con el valor entero constante especificado. La directiva

```
#line 100
```

inicia la numeración de líneas a partir de 100, empezando con la siguiente línea de código fuente. En la directiva **#line** puede incluirse un nombre de archivo. La directiva

```
#line 100 "file1.c"
```

indica que las líneas están numeradas a partir de 100 empezando desde la siguiente línea de código fuente, y el nombre del archivo para fines de cualquier mensaje de compilador es "file1.c". Esta directiva normalmente se utiliza para ayudar a preparar los mensajes producidos debido a errores de sintaxis y a advertencias más significativas del compilador. En el archivo fuente no aparecen los números de línea.

13.9 Constantes simbólicas predefinidas

Existen cinco *constantes simbólicas predefinidas* (figura 13.1). Los identificadores correspondientes a cada una de las constantes simbólicas predefinidas empiezan y terminan con *dos* subrayados. Estos identificadores y el identificador `defined` (utilizado en la sección 13.5) no pueden ser utilizados en las directivas `#define` o `#undef`.

13.10 Asertos

La *macro* `assert` definida en el archivo de cabecera `assert.h`, prueba el valor de una expresión. Si el valor de la expresión es 0 (falso), entonces `assert` imprime un mensaje de error, y llama a la función `abort` (de la biblioteca general de utilerías `stdlib.h`) para terminar la ejecución del programa. Esta es una herramienta de depuración útil, para probar si una variable tiene el valor correcto. Por ejemplo, suponga que en un programa la variable `x` jamás debería ser mayor de 10. Se podría utilizar una afirmación para probar el valor de `x` y si el valor de `x` es incorrecto imprimir un mensaje de error. El enunciado sería

```
assert(x <= 10);
```

Si cuando se encuentre este enunciado en un programa `x` es mayor que 10, se imprimirá un mensaje de error, conteniendo el número de línea y el nombre del archivo, y el programa terminará. Entonces el programador podrá concentrarse en esta área del código y encontrar el error. Si queda definida la constante simbólica `NDEBUG`, asertos subsecuentes serán ignoradas. Entonces, una vez que ya no sean requeridas los asertos, la línea

Constante simbólica	Explicación
<code>__LINE__</code>	Número de línea de la línea actual del código fuente (una constante entera).
<code>__FILE__</code>	El nombre supuesto del archivo fuente (una cadena).
<code>__DATE__</code>	La fecha de compilación del archivo fuente (una cadena de la forma "Mmm dd yyyy como "Jan 19 1991").
<code>__TIME__</code>	La hora en que fue compilado el archivo fuente (una literal de cadena de la forma "hh:mm:ss").
<code>__STDC__</code>	La constante entera 1. Se utiliza para indicar que ésta puesta en marcha cumple con ANSI.

Fig. 13.1 Las constantes simbólicas predefinidas.

```
#define NDEBUG
```

es inserta en el archivo del programa, en vez de borrar de manera manual cada una de los asertos.

Resumen

- Todas las directivas de preprocesador empiezan con `#`.
- Sólo caracteres de espacio en blanco pueden aparecer en una línea antes de una directiva de preprocesador.
- La directiva `#include` incluye una copia del archivo especificado. Si el nombre del archivo está encerrado entre comillas, el preprocesador empieza buscando el archivo a incluirse en el mismo directorio que el archivo que se está compilando. Si el nombre del archivo está encerrado en corchetes angulares (`<` y `>`), la búsqueda se ejecuta de una forma definida de acuerdo a la puesta en marcha.
- La directiva de preprocesador `#define` se utiliza para crear constantes simbólicas y macros.
- Una constante simbólica es el nombre para una constante.
- Una macro es una operación definida en una directiva de preprocesador `#define`. Las macros pueden ser definidas con o sin argumentos.
- El texto de remplazo para una macro o una constante simbólica es cualquier texto que quede en la línea después del identificador en la directiva `#define`. Si el texto de remplazo para una macro o una constante simbólica es más largo que el resto de la línea, se coloca una diagonal invertida (`\`) al final de la línea, lo que indica que el texto de remplazo continúa sobre la línea siguiente.
- Las constantes simbólicas y las macros pueden ser descartadas utilizando la directiva de preprocesador `#undef`. La directiva `#undef` "elimina la definición" de la constante simbólica o del nombre de la macro.
- El alcance de una constante simbólica o de una macro es desde su definición hasta que queda eliminada su definición mediante `#undef`, o si no hasta el final del archivo.
- La compilación condicional le permite al programador controlar la ejecución de las directivas de preprocesador y la compilación del código de programa.
- Las directivas de preprocesador condicionales evalúan expresiones enteras constantes. Las expresiones de cambio de tipo, las expresiones `sizeof` y las constantes de enumeración no pueden ser evaluadas en directivas de preprocesador.
- Cada constructor `#if` termina con `#endif`.
- Las directivas `#ifdef` y `#ifndef` han sido diseñadas como abreviaturas de `#if defined(nombre)` y `#if !defined(nombre)`.
- Un constructor de preprocesador condicional de varias partes puede ser probado utilizando `#elif` (el equivalente de `else if` en una estructura `if`) y `#else` (el equivalente de `else` en una estructura `if`) directivas.
- La directiva `#error` imprime un mensaje, que depende de la puesta en práctica, y que incluye las componentes léxicas especificadas en la directriz.
- La directiva `#pragma` genera una acción definida por la puesta en práctica. Si el `pragma` no es reconocido por la puesta en marcha, será ignorado.

- El operador # hace que un componente léxico de texto de remplazo, se convierta a una cadena encerrada entre comillas. El operador # debe ser utilizado en una macro con argumentos, porque el operando # debe ser un argumento de la macro.
- El operador ## concatena dos componentes léxicos. El operador ## debe de tener dos operandos.
- La directiva de preprocesador #line hace que las líneas de código fuente subsecuentes vuelvan a ser numeradas, iniciándose a partir del valor entero constante especificado.
- Existen cinco constantes simbólicas predefinidas. La constante __LINE__ es el número de línea de la línea de código fuente actual (un entero). La constante __FILE__ es el nombre supuesto del archivo (una cadena). La constante __DATE__ es la fecha de compilación del archivo fuente (una cadena). La constante __TIME__ es la hora de compilación del archivo fuente (una cadena). La constante __STDC__ es 1; su propósito es indicar que la instalación cumple con ANSI. Note que cada una de las constantes simbólicas predefinidas empieza y termina con dos subrayados.
- La macro assert definida en el archivo de cabecera assert.h, —prueba el valor de una expresión. Si el valor de la expresión es 0 (falsa), entonces assert imprime un mensaje de error y llama a la función abort para terminar la ejecución del programa.

Terminología

#define	assert
#elif	assert.h
#else	preprocesador C
#endif	comando cc en UNIX
#error	operador ## de concatenación de preprocesador
#if	compilación condicional
#ifdef	directrices de preprocesador de ejecución condicional
#ifndef	operador # de preprocesador de conversión a cadena
#include <filename>	depurador
#include "filename"	expandir una macro
#line	archivo de cabecera
#pragma	macro
#undef	macro con argumentos
\ (diagonal invertida) carácter de continuación	constantes simbólicas predefinidas
__DATE__	directiva de preprocesador
__FILE__	texto de remplazo
__LINE__	alcance de una constante simbólica o de una macro
__STDC__	archivos de cabecera de la biblioteca estándar
__TIME__	stdio.h
a. out en UNIX	stdlib.h
abort	constante simbólica
argumento	

Errores comunes de programación

13.1 Olvidar encerrar los argumentos de macro en paréntesis en el texto de remplazo.

13.2 Insertar enunciados printf compilados condicional para fines de depuración, en posiciones donde en ese momento C espera un enunciado. En este caso, el enunciado compilado condicional debería haberse encerrado en un enunciado compuesto. Entonces, cuando el programa se compile con enunciados de depuración, el flujo del control del programa no será alterado.

Práctica sana de programación

13.1 Utilizar nombres significativos para las constantes simbólicas, ayuda a que los programas resulten más autodocumentados.

Sugerencia de rendimiento

13.1 A veces las macros pueden ser utilizadas para substituir una llamada de función por código en línea, previo al tiempo de ejecución. Esto elimina la sobrecarga de una llamada de función.

Ejercicios de autoevaluación

- 13.1 Llene los espacios en blanco con cada uno de los siguientes:
- a) Cada directiva de preprocesador deberá empezar con _____.
 - b) El constructor de compilación condicional puede ser extendido para probar casos múltiples utilizando las directivas _____ y _____.
 - c) La directiva _____ genera macros y constantes simbólicas.
 - d) Sólo caracteres _____ pueden aparecer sobre una línea antes de una directiva de preprocesador.
 - e) La directiva _____ descarta constantes simbólicas y nombres de macros.
 - f) Las directivas _____ y _____ son una notación abreviada en lugar de #if defined (nombre) y #if !defined (nombre).
 - g) _____ le permite al programador controlar la ejecución de las directivas de preprocesador, y la compilación del código del programa.
 - h) La macro _____ imprime un mensaje y termina la ejecución de un programa si el valor de la expresión que la macro evalúa es 0.
 - i) La directiva _____ inserta un archivo en otro archivo.
 - j) El operador _____ concatena sus dos argumentos.
 - k) El operador _____ convierte su operando a una cadena.
 - l) El carácter _____ indica que el texto de remplazo de una constante simbólica o de una macro continúa en la línea siguiente.
 - m) La directiva _____ hace que las líneas de código fuente se enumeren a partir del valor indicado, empezando con la siguiente línea de código fuente.
- 13.2 Escriba un programa que imprima los valores de las constantes simbólicas predefinidas, listadas en la figura 13.1.
- 13.3 Escriba una directiva de preprocesador para que lleve a cabo cada una de las siguientes:
- a) Definir la constante simbólica YES para que tenga el valor 1.
 - b) Definir la constante simbólica NO para que tenga el valor 0.
 - c) Incluir el archivo de cabecera common.h. El archivo de cabecera se encuentra en el mismo directorio que el archivo bajo compilación.
 - d) Renumerar las líneas restantes del archivo, empezando con el número de línea 3000.
 - e) Si está definida la constante simbólica TRUE, elimine su definición, y vuélvala a definir como 1. No utilice #ifdef.
 - f) Si la constante simbólica TRUE está definida, elimine su definición, y vuélvala a definir como 1. Utilice la directiva de preprocesador #ifdef.

- g) Si la constante simbólica **TRUE** no es igual a 0, defina la constante simbólica **FALSE** como 0. De lo contrario defina **FALSE** como 1.
- h) Defina la macro **SQUARE_VOLUME**, que calcula el volumen de un cuadrado. La macro toma un argumento.

Respuesta a los ejercicios de autoevaluación

- 13.1 a) **#**. b) **#elif**, **#else**. c) **#define**. d) espacio en blanco. e) **#undef**. f) **#ifdef**, **#ifndef**. g) compilación condicional. h) **assert**. i) **#include**. j) **##**. k) **#**. l) ****. m) **#line**.

- 13.2

```
/* Print the values of the predefined macros */
#include <stdio.h>
main()
{
    printf("__LINE__ = %d\n", __LINE__);
    printf("__FILE__ = %s\n", __FILE__);
    printf("__DATE__ = %s\n", __DATE__);
    printf("__TIME__ = %s\n", __TIME__);
    printf("__STDC__ = %d\n", __STDC__);
}
```

```
__LINE__ = 5
__FILE__ = macros.c
__DATE__ = Sep 08 1993
__TIME__ = 10:23:47
__STDC__ = 1
```

- 13.3 a) **#define YES 1**
 b) **#define NO 0**
 c) **#include "common.h"**
 d) **#line 3000**
 e) **#if defined(TRUE)**
 #undef TRUE
 #define TRUE 1
 #endif
 f) **#ifdef TRUE**
 #undef TRUE
 #define TRUE 1
 #endif
 g) **#if TRUE**
 #define FALSE 0
 #else
 #define FALSE 1
 #endif
 h) **#define SQUARE_VOLUME(x) (x) * (x) * (x)**

Ejercicios

- 13.4 Escriba un programa que defina una macro con un argumento para calcular el volumen de una esfera. El programa deberá calcular el volumen para esferas de radios de 1 a 10, e imprimir los resultados en formato tabular. La fórmula del volumen de una esfera es:

$$(4/3) * \pi * r^3$$

donde π es 13.14159

- 13.5 Escriba un programa que produzca la siguiente salida:

```
The sum of x and y is 13
```

El programa deberá definir la macro **SUM** con dos argumentos, **x** y **y**, y utilizar **SUM** para producir la salida.

- 13.6 Escriba un programa que utilice la macro **MINIMUM2** para determinar el más pequeño de dos valores numéricos. Introduzca los valores desde el teclado.

- 13.7 Escriba un programa que utilice la macro **MINIMUM3** para determinar el más pequeño de tres valores numéricos. La macro **MINIMUM3** deberá utilizar la macro **MINIMUM2** definida en el ejercicio 13.6 para determinar el valor más pequeño. Introduzca los valores desde el teclado.

- 13.8 Escriba un programa que utilice la macro **PRINT** para imprimir un valor de cadena.

- 13.9 Escriba un programa que utilice la macro **PRINTARRAY** para imprimir un arreglo de enteros. La macro deberá recibir como argumentos el arreglo y el número de elementos en el arreglo.

- 13.10 Escriba un programa que utilice la macro **SUMARRAY** para sumar los valores de un arreglo numérico. La macro deberá recibir como argumentos el arreglo y el número de elementos en el arreglo.

14

Temas avanzados

Objetivos

- Ser capaz de redirigir la entrada de teclado para que provenga de un archivo.
- Ser capaz de redirigir la salida a pantalla a un archivo.
- Ser capaz de escribir funciones que utilicen listas de argumentos de longitud variable.
- Ser capaz de procesar argumentos en la línea de comandos.
- Ser capaz de asignar tipos específicos a constantes numéricas.
- Ser capaz de utilizar archivos temporales.
- Ser capaz de procesar eventos inesperados dentro de un programa.
- Ser capaz de asignar memoria dinámicamente para los arreglos.
- Ser capaz de cambiar el tamaño de la memoria anteriormente dinámica asignada.

*Utilizaremos una señal que ya he probado y comprobado
llega lejos y es fácil de gritar. ¡Waa-hoo!*

Zane Grey

*Uselo, gástelo;
aprovéchelo o descártelo.*

Anónimo

Verdaderamente es un problema de tres pipas.

Sir Arthur Conan Doyle

Sinopsis

- 14.1 Introducción
- 14.2 Cómo redirigir entradas/salidas en sistemas UNIX y DOS
- 14.3 Listas de argumentos de longitud variable
- 14.4 Cómo utilizar argumentos en la línea de comandos
- 14.5 Notas sobre la compilación de programas formados por múltiples archivos fuente
- 14.6 Terminación de programas mediante Exit y Atexit
- 14.7 El calificador de tipo volátil
- 14.8 Sufijos para constantes enteras y de punto flotante
- 14.9 Más sobre archivos
- 14.10 Manejo de señales
- 14.11 Asignación dinámica de memoria: funciones Calloc y Realloc
- 14.12 La bifurcación incondicional: Goto

Resumen • Terminología • Errores comunes de programación • Sugerencias de portabilidad • Sugerencias de rendimiento • Observaciones de ingeniería de software • Ejercicios de autoevaluación • Respuestas a los ejercicios de autoevaluación • Ejercicios.

14.1 Introducción

En esta capítulo se presentan varios temas avanzados, por lo regular no se tratan en cursos introductorios. Muchas de las capacidades que aquí se analizan son específicas a sistemas operativos particulares, en especial a UNIX y/o a DOS.

14.2 Cómo redirigir entradas/salidas en sistemas UNIX y DOS

Normalmente la entrada a un programa es a partir del teclado (entrada estándar), y la salida de un programa se muestra en la pantalla (salida estándar). En la mayor parte de los sistemas de computación y en particular en los sistemas UNIX y DOS es posible *redirigir* las entradas, para que en vez del teclado provengan de un archivo, y redirigir las salidas, para que en vez de en la pantalla sean colocadas en un archivo. Ambas formas de redirección pueden ser llevadas a cabo sin tener que utilizar las capacidades de procesamiento de archivos de la biblioteca estándar.

A partir de la línea de comandos UNIX existen varias maneras para redirigir la entrada y la salida. Considere al archivo ejecutable **sum**, que introduce enteros uno por uno, y lleva un total acumulativo de los valores hasta que se define el indicador de fin de archivo, y a continuación imprime el resultado. Por lo regular el usuario introduce enteros a partir del teclado y escribe la combinación de teclas de fin de archivo, para indicar que no se introducirán más valores. Con la redirección de la entrada, la entrada puede estar almacenada en un archivo. Por ejemplo, si los datos están almacenados en el archivo **input**, la línea de comando

```
$ sum < input
```

hace que se ejecute el programa **sum**; el *símbolo de redirección de entrada* (<) indica que los datos en el archivo **input** deben ser utilizados como entrada para el programa. En un sistema DOS la redirección de la entrada se efectúa en forma idéntica.

Note que en UNIX la indicación de la línea de comandos es el signo \$ (algunos sistemas UNIX utilizan la indicación %). Los estudiantes encuentran a menudo difícil comprender que la redirección es una función del sistema operativo, y no otra característica de C.

El segundo método para redirigir la entrada es *el entubamiento*. Una *tubería* (|) hace que la salida de un programa sea redirigida como entrada de otro programa. Suponga que el programa **random** tiene como salida una serie de enteros al azar; la salida de **random** puede ser “entubada” en forma directa al programa **sum**, utilizando la línea de comandos UNIX

```
$ random | sum
```

Esto hace que se calcule la suma de los enteros producidos por **random**. El entubamiento puede ser ejecutado en UNIX y en DOS.

La salida de programa puede ser redirigida a un archivo, mediante el uso del símbolo de *redirección de salida* (>) (tanto en UNIX como en DOS se utiliza el mismo símbolo). Por ejemplo, para redirigir la salida del programa **random** al archivo **out**, utilice

```
$ random > out
```

Por último, la salida del programa puede ser agregada al final de un archivo existente, utilizando el *símbolo de agregar salida* (>>) (tanto en UNIX como en DOS se utiliza el mismo símbolo). Por ejemplo, para agregar la salida del programa **random** al archivo **out**, creado en línea de comando anterior, utilice la línea de comando

```
$ random >> out
```

14.3 Listas de argumentos de longitud variable

Es posible crear funciones que reciban un número no especificado de argumentos. La mayor parte de los programas en el texto han utilizado la función **printf** estándar de biblioteca la cual, como usted sabe, toma un número variable de argumentos. Como mínimo, **printf** debe recibir una cadena como primer argumento, pero **printf** puede recibir cualquier número de argumentos adicionales. El prototipo de función correspondiente a **printf** es

```
int printf(const char *format, ...);
```

En el prototipo de función los puntos suspensivos (...) indican que la función recibe un número variable de argumentos de cualquier tipo. Note que los puntos suspensivos deberán siempre estar colocados al final de la lista de parámetros.

Los macros y definiciones del *encabezado de argumentos variables* **stdarg.h** (figura 14.1) proveen las capacidades necesarias para construir funciones con listas de argumentos de longitud variable. El programa de la figura 14.2 demuestra una función **average**, que recibe un número variable de argumentos. El primer argumento de **average** es siempre el número de valores a promediarse.

INSTITUTO VENEZOLANO DE LA REPÚBLICA
 FACULTAD DE INGENIERÍA
 DEPARTAMENTO DE

Identificador	Explicación
va_list	Un tipo utilizable para conservar información necesaria por las macros va_start , va_arg y va_end . Para tener acceso a los argumentos en una lista de argumentos de longitud variable, debe declararse un objeto del tipo va_list .
va_start	Una macro que se invoca antes de tener acceso a los argumentos de una lista de argumentos de longitud variable. La macro inicializa el objeto declarado mediante va_list , para que sea utilizada con las macros va_arg y va_end .
va_arg	Una macro que se expande a una expresión del valor y del tipo del siguiente argumento, existente en la lista de argumentos de longitud variable. Cada invocación de va_arg modifica el objeto declarado con va_list , de tal forma que el objeto señale al siguiente argumento dentro de la lista.
va_end	Una macro que facilita el regreso normal de una función cuya lista de argumentos de longitud variable ha sido referenciada por la macro va_start .

Fig. 14.1 El tipo y las macros definidas en el encabezado `stdarg.h`.

La función **average** utiliza todas las definiciones y macros del encabezado **stdarg.h**. El objeto **ap**, del tipo **va_list**, es utilizado por las macros **va_start**, **va_arg** y **va_end** para procesar la lista de argumentos de longitud variable de la función **average**. La función empieza invocando la macro **va_start** para inicializar el objeto **ap** para su uso en **va_arg** y **va_end**. La macro recibe dos argumentos —el objeto **ap** y el identificador al argumento más a la derecha dentro de la lista de argumentos, antes de los puntos suspensivos— en este caso **i** (aquí **va_start** utilizará **i** para determinar dónde empieza la lista de argumentos de longitud variable). A continuación la función **average** añade en forma repetida los argumentos en la lista de argumentos de longitud variable de la variable **total**. El valor a añadirse a **total** se toma de la lista de argumentos invocando la macro **va_arg**. La macro **va_arg** recibe dos argumentos: el objeto **ap**, y el tipo de valor esperado en la lista de argumentos, en este caso **double**. La macro **regresa** el valor del argumento. La función **average** invoca la macro **va_end**, con el objeto **ap** como argumento, para facilitar un regreso normal hacia **main** a partir de **average**. Por último, se calcula el promedio y se **regresa** a **main**.

Error común de programación 14.1

Colocar puntos suspensivos en la mitad de una lista de parámetros de función. Los puntos suspensivos sólo pueden ser colocados al final de la lista de parámetros.

El lector puede cuestionar cómo pueden saber las funciones **printf** y **scanf** qué tipo utilizar en cada macro **va_arg**. La respuesta es que **printf** y **scanf** rastrean los especificadores de conversión de formato existentes en la cadena de control de formato, a fin de determinar el tipo del siguiente argumento a procesar.

```
/* Using variable-length argument lists */

#include <stdio.h>
#include <stdarg.h>

double average(int, ...);

main()
{
    double w = 37.5, x = 22.5, y = 1.7, z = 10.2;

    printf("%s%.1f\n%s%.1f\n%s%.1f\n%s%.1f\n\n",
           "w = ", w, "x = ", x, "y = ", y, "z = ", z);
    printf("%s%.3f\n%s%.3f\n%s%.3f\n",
           "The average of w and x is ",
           average(2, w, x),
           "The average of w, x, and y is ",
           average(3, w, x, y),
           "The average of w, x, y, and z is ",
           average(4, w, x, y, z));

    return 0;
}

double average(int i, ...)
{
    double total = 0;
    int j;
    va_list ap;

    va_start(ap, i);

    for (j = 1; j <= i; j++)
        total += va_arg(ap, double);

    va_end(ap);
    return total / i;
}
```

w = 37.5
x = 22.5
y = 1.7
z = 10.2

The average of w and x is 30.000
The average of w, x, and y is 20.567
The average of w, x, y, and z is 17.975

Fig. 14.2 Cómo utilizar listas de argumentos de longitud variable.

14.4 Cómo utilizar argumentos en la línea de comandos

En muchos sistemas y en particular en DOS y en UNIX incluyendo los parámetros `int argc` y `char *argv[]` en la lista de parámetros de `main`, es posible pasar argumentos a `main` a partir de la línea de comandos. El parámetro `argc` recibe el número de argumentos de la línea de comando. El parámetro `argv` es un arreglo de cadenas, en el cual se almacenan los argumentos de la línea de comando reales. Usos comunes de los argumentos en la línea de comandos incluyen la impresión de los mismos, pasar opciones a un programa, y pasar nombres de archivo a un programa.

El programa de la figura 14.3 copia un archivo a otro archivo, carácter por carácter. El archivo ejecutable para el programa se llama `copy`. En un sistema UNIX una línea de comando típica para el programa `copy` es

```
$ copy input output
```

Esta línea de comando indica que el archivo `input` será copiado al archivo `output`. Cuando se ejecuta el programa, si `argc` no es 3 (`copy` cuenta como uno de los argumentos), el programa imprime un mensaje de error y termina. De lo contrario, `argv` contiene las cadenas "`copy`", "`input`", y "`output`". Los argumentos segundos y terceros de la línea de comandos se utilizan como nombres de archivo por el programa. Los archivos se abren utilizando la función `fopen`. Si ambos archivos se abren con éxito, los caracteres se leen del archivo `input` y se escriben al archivo `output` hasta que el indicador de fin de archivo del archivo `input` queda definido. Entonces termina el programa. El resultado es una copia exacta del archivo `input`. Note que no todos los sistemas de cómputo aceptan argumentos en la línea de comandos tan fácil como UNIX y DOS. Por ejemplo, los sistemas Macintosh y VMS, requieren de ajustes especiales para el proceso de argumentos en la línea de comandos. Vea los manuales de su sistema para más información relativa a argumentos en la línea de comandos.

14.5 Notas sobre la compilación de programas con varios archivos fuente

Como se indicó en el texto, es posible construir programas que estén formados por múltiples archivos fuente (vea el capítulo 16, "Clases"). Existen varias consideraciones a tomar en cuenta al crear programas en múltiples archivos. Por ejemplo, la definición de una función deberá estar por completo contenida en un archivo —no puede estar contenida en dos o más archivos.

En el capítulo 5, presentamos los conceptos de clase de almacenamiento y alcance. Aprendimos que las variables declaradas por afuera de cualquier definición de función son por omisión de clase de almacenamiento estática, y se conocen como variables globales. Una vez declaradas, las variables globales son accesibles a cualquier función definida en el mismo archivo. Las variables globales también son accesibles a funciones en otros archivos, pero sin embargo, estas variables globales deberán ser declaradas en cada uno de los archivos en los cuales serán utilizadas. Por ejemplo, si en un archivo definimos la variable entera global `flag`, y en un segundo archivo nos referimos a ella, el segundo archivo deberá contener la declaración.

```
extern int flag;
```

antes del uso de la variable en dicho archivo. En la declaración anterior, el especificador de clase de almacenamiento `extern` le indica al compilador que la variable `flag` está definida más adelante en el mismo archivo o en uno distinto. El compilador informa al enlazador que en dicho archivo aparecen referencias a la variable `flag` sin resolver (el compilador no sabe donde `flag`

```
/* Using command-line arguments */
#include <stdio.h>

main(int argc, char *argv[])
{
    FILE *inFilePtr, *outFilePtr;
    int c;

    if (argc != 3)
        printf("Usage: copy infile outfile\n");
    else
        if ((inFilePtr = fopen(argv[1], "r")) != NULL)

            if ((outFilePtr = fopen(argv[2], "w")) != NULL)

                while ((c = fgetc(inFilePtr)) != EOF)
                    fputc(c, outFilePtr);

            else
                printf("File \"%s\" could not be opened\n", argv[2]);

        else
            printf("File \"%s\" could not be opened\n", argv[1]);

    return 0;
}
```

Fig. 14.3 Cómo utilizar argumentos en la línea de comandos.

queda definida, por lo que le deja al enlazador intentar encontrar a `flag`). Si el enlazador no puede localizar la definición de `flag`, se genera un error de enlace, y no se produce un archivo ejecutable. Si el enlazador localiza una definición global apropiada, éste resuelve las referencias indicando donde está localizada `flag`.

Sugerencia de rendimiento 14.1

Las variables globales aumentan el rendimiento debido a que son de manera directa accesibles por cualquier función —se elimina la sobrecarga de pasar datos a función.

Observación de ingeniería de software 14.1

Las globales variables deberían de evitarse, a menos de que el rendimiento de la aplicación sea crítico, porque violan el principio del mínimo privilegio.

Igual que las declaraciones `extern` pueden ser utilizadas para declarar variables globales en otros archivos de programa, los prototipos de función pueden extender el alcance de una función más allá del archivo en el cual ha sido definida (en un prototipo de función no es requerido el especificador `extern`). Esto se lleva a cabo incluyendo el prototipo de función en cada uno de los archivos en los cuales dicha función fue invocada, y compilando ambos archivos juntos (vea la sección 13.2). Los prototipos de función le indican al compilador que la función especificada se define más adelante en el mismo archivo o en un archivo distinto. De nuevo, el compilador no intenta resolver referencias a dicha función esa tarea se le deja al enlazador. Si el enlazador no puede localizar una definición de función apropiada, se genera un error.

Como ejemplo del uso de prototipos de función para extender el alcance de una función, considere cualquier programa que contenga la directiva de preprocesador `#include <stdio.h>`. Esta directiva incluye en un archivo los prototipos de función para funciones tales como `printf` y `scanf`. Otras funciones en el archivo pueden utilizar `printf` y `scanf` para llevar a cabo sus tareas. Las funciones `printf` y `scanf` se definen para nosotros en forma independiente. No necesitamos saber dónde están definidas. Estamos sólo volviendo a utilizar dicho código en nuestros programas. El enlazador resuelve nuestras referencias a dichas funciones en forma automática. Este proceso nos permite utilizar las funciones estándar de biblioteca.

Observación de ingeniería de software 14.2

La creación de programas en múltiples archivos fuente facilita la reutilización del software y buena ingeniería de software. Las funciones pueden ser comunes para muchas aplicaciones. En casos como éstos, estas funciones deberán ser almacenadas en sus propios archivos fuente, y cada archivo fuente deberá tener su correspondiente archivo de cabecera, que contenga los prototipos de función. Esto permite a los programadores de distintas aplicaciones para reutilizar el mismo código, mediante la inclusión del archivo de cabecera apropiado, y compilar su aplicación con el archivo fuente correspondiente.

Sugerencia de portabilidad 14.1

Algunos sistemas no aceptan nombres de variables globales o nombres de funciones con más de 6 caracteres. Esto deberá ser tomado en cuenta al escribir programas que se planea serán transportados a varias plataformas.

Es posible restringir el alcance de una variable global o de una función al archivo en el cual está definido. El especificador de clase de almacenamiento `static`, al ser aplicado a una variable global o a una función, impide que sea utilizada por cualquier función que no quede definida dentro del mismo archivo. Esto se conoce como *enlace interno*. Las variables globales y las funciones que en sus definiciones no estén precedidas por `static` tienen *enlace externo* —se puede tener acceso a ellas desde otros archivos, si dichos archivos contienen las declaraciones apropiadas y/o los prototipos de función.

La declaración de variable global

```
static float pi = 3.14159;
```

crea la variable `pi` del tipo `float`, la inicializa a `3.14159`, e indica que `pi` es conocida sólo por las funciones dentro del archivo en la cual está definida.

El especificador `static` se utiliza por lo general con funciones de utilidad, que sólo son llamadas por funciones de un archivo en particular. Si fuera de un archivo en particular una función no es requerida, deberá cumplirse mediante el uso de `static` con el principio del mínimo privilegio. Si en un archivo una función se define antes de su uso, `static` deberá ser aplicado a la definición de función. De lo contrario, `static` deberá aplicarse a el prototipo de función.

Al construir programas extensos en múltiples archivos fuente, la compilación del programa se convierte en tediosa si se hacen pequeñas modificaciones a un archivo y todo el programa debe ser vuelto a compilar. Muchos sistemas tienen utilerías especiales, que recopilan sólo el archivo de programa modificado. En sistemas UNIX esta utilidad se denomina `make`. La utilidad `make` lee un archivo llamado `makefile`, que contiene instrucciones para compilar y enlazar el programa. Los sistemas como Borland C++ y Microsoft C/C++ 7.0 para PC también ofrecen utilerías `make`. Para mayor información sobre las utilerías `make`, vea el manual correspondiente a su sistema particular.

14.6 Terminación de programa mediante `Exit` y `Atexit`

La biblioteca general de utilerías (`stdlib.h`) contiene métodos de terminar la ejecución de programa distintos al regreso convencional a partir de la función `main`. La función `exit` obliga a que se termine un programa, como si se hubiera ejecutado en forma normal. Esta función se utiliza a menudo para terminar un programa cuando se detecta un error en la entrada o si un archivo a procesarse por el programa no puede ser abierto. La función `atexit` registra en el programa, una función, que a la terminación exitosa del programa será llamada, es decir, ya sea cuando termina el programa llegando al final de `main`, o cuando se invoca a `exit`.

La función `atexit` toma como argumento un apuntador a una función (es decir, el nombre de la función). Las funciones llamadas a la terminación del programa no pueden tener argumentos, y no pueden regresar un valor. Se pueden registrar hasta 32 funciones para ejecución a la terminación del programa.

La función `exit` toma un argumento. El argumento por lo regular es o la constante simbólica `EXIT_SUCCESS` o la constante simbólica `EXIT_FAILURE`. Si `exit` es llamada mediante `EXIT_SUCCESS`, el valor definido por la puesta en marcha para una terminación exitosa es regresado al entorno llamador. Si `exit` es llamado mediante `EXIT_FAILURE`, el valor definido por la puesta en marcha correspondiente a una terminación no exitosa es regresado. Cuando se invoca la función `exit`, cualesquiera funciones registradas ya con `atexit` son invocadas en orden inverso de su registro, son vaciados y cerrados todos los flujos asociados con el programa, y el control regresa al entorno huésped. El programa de la figura 14.4 prueba las funciones `exit` y `atexit`. El programa solicita al usuario que determine si el programa debe terminarse con `exit` o llegando al final de `main`. Note que en cada caso la función `print` se ejecuta a la terminación del programa.

14.7 El calificador de tipo volátil

En los capítulos 6 y 7, presentamos el calificador de tipo `const`. ANSI C también dispone del calificador de tipo `volatile`. El estándar ANSI (An90) indica que cuando se utiliza `volatile` para calificar un tipo, la naturaleza del acceso a un objeto de dicho tipo dependerá de la puesta en marcha. Kernighan y Ritchie (Ke88) indican que el calificador `volatile` se utiliza para suprimir varios tipos de optimizaciones.

14.8 Sufijos para constantes enteras y de punto flotante

C contiene sufijos enteros y de punto flotante para especificar los tipos de las constantes enteras y de punto flotante. Los sufijos enteros son: `u` o bien `U` para un entero `unsigned`, `l` o `L` para un entero `long` y `ul` o `UL` para un entero `unsigned long`. Las siguientes constantes son del tipo `unsigned`, `long` y `unsigned long` respectivamente:

```
174u
8358L
28373ul
```

Si una constante entera no tiene sufijo, su tipo queda determinado con el primer tipo capaz de almacenar un valor de dicho tamaño (primero `int`, después `long int`, y por último `unsigned long int`).

Los sufijos de punto flotante son: `f` o `F` para un `float`, y `l` o `L` para un `long double`. Las siguientes constantes son del tipo `long double` y `float`, respectivamente:

```
/* Using the exit and atexit functions */

#include <stdio.h>
#include <stdlib.h>

void print(void);

main()
{
    int answer;

    atexit(print);      /* register function print */
    printf("Enter 1 to terminate program with function exit\n"
           "Enter 2 to terminate program normally\n");
    scanf("%d", &answer);

    if (answer == 1) {
        printf("\nTerminating program with function exit\n");
        exit(EXIT_SUCCESS);
    }

    printf("\nTerminating program by reaching the end of main\n");
    return 0;
}

void print(void)
{
    printf("Executing function print at program termination\n"
           "Program terminated\n");
}
```

Enter 1 to terminate program with function exit
Enter 2 to terminate program normally
: 1

Terminating program with function exit
Executing function print at program termination
Program terminated

Enter 1 to terminate program with function exit
Enter 2 to terminate program normally
: 2

Terminating program by reaching the end of main
Executing function print at program termination
Program terminated

Fig. 14.4 Cómo utilizar las funciones `exit` y `atexit`.

3.14159L
1.28f

Una constante de punto flotante sin sufijo automáticamente es del tipo `double`.

14.9 Más sobre archivos

En el capítulo 11, se presentaron las capacidades para procesar archivos de texto mediante acceso secuencial y acceso directo. C también proporciona capacidades para el proceso de archivos binarios, pero algunos sistemas de cómputo no aceptan archivos binarios. Si no son aceptados los archivos binarios, y un archivo es abierto en modo de archivo binario (figura 14.5), el archivo será procesado como un archivo de texto. Los archivos binarios deberán ser utilizados en lugar de los archivos de texto, sólo en aquellas situaciones en que las condiciones de velocidad, almacenamiento y/o compatibilidad demanden archivos binarios. De lo contrario, siempre los archivos de texto deberán preferirse, debido a su inherente portabilidad, y debido a la posibilidad de utilizar otras herramientas estándar para examinar y manipular los datos del archivo.

Sugerencia de rendimiento 14.2

Considere utilizar archivos binarios en lugar de archivos de texto en aquellas aplicaciones que exijan alto rendimiento

Modo	Descripción
rb	Abre un archivo binario para lectura
wb	Crea un archivo binario para escritura. Si el archivo ya existe, descartará el contenido actual.
ab	Agrega; abre o crea un archivo binario para escritura al final del archivo.
rb+	Abre un archivo binario para actualizar (lectura y escritura).
wb+	Crea un archivo binario para actualizar. Si el archivo ya existe, descartará el contenido actual.
ab+	Agrega; abre o crea un archivo binario para actualizar; toda la escritura se efectuará al final del archivo.

Fig. 14.5 Modos de archivo binario abierto.

Sugerencia de portabilidad 14.2

Cuando esté escribiendo programas portátiles utilice archivos de texto.

Además de las funciones de procesamiento de archivos analizadas en el capítulo 11, la biblioteca estándar también tiene la función `tmpfile`, que abre un archivo temporal en modo `"wb+"`. Aunque este es un modo de archivo binario, algunos sistemas procesan los archivos temporales como archivos de texto. Un archivo temporal existe sólo hasta que es cerrado con `fclose`, o bien hasta que el programa termina.

El programa de la figura 14.6 sustituye los tabuladores existentes en un archivo por espacios. El programa solicita al usuario que escriba el nombre del archivo a modificarse. Si tanto el archivo

escrito por el usuario como el archivo temporal son abiertos con éxito, el programa lee caracteres del archivo a modificarse, y los escribe en el archivo temporal. Si el carácter leído es un tabulador ('`\t`'), será remplazado por un espacio y será escrito al archivo temporal. Cuando se llegue al fin del archivo bajo modificación, utilizando `rewind` los apuntadores de archivo para cada archivo serán reposicionados al principio de cada uno. A continuación, el archivo temporal se copia al archivo original, carácter por carácter. Con el fin de confirmar que los caracteres escritos son correctos, el programa imprime el archivo original conforme copia caracteres al archivo temporal, así como imprime el nuevo archivo conforme copia caracteres del archivo temporal al archivo original.

```
/* Using temporary files */
#include <stdio.h>

main()
{
    FILE *filePtr, *tempFilePtr;
    int c;
    char fileName[30];

    printf("This program changes tabs to spaces.\n");
    printf("Enter a file to be modified: ");
    scanf("%s", fileName);

    if ( ( filePtr = fopen(fileName, "r+") ) != NULL)

        if ( ( tempFilePtr = tmpfile() ) != NULL) {
            printf("\nThe file before modification is:\n");

            while ( ( c = getc(filePtr) ) != EOF) {
                putchar(c);
                putc(c == '\t' ? ' ': c, tempFilePtr);
            }

            rewind(tempFilePtr);
            rewind(filePtr);
            printf("\n\nThe file after modification is:\n");

            while ( ( c = getc(tempFilePtr) ) != EOF) {
                putchar(c);
                putc(c, filePtr);
            }

        }
        else
            printf("Unable to open temporary file\n");
    else
        printf("Unable to open %s\n", fileName);

    return 0;
}
```

Fig. 14.6 Cómo utilizar los archivos temporales (parte 1 de 2).

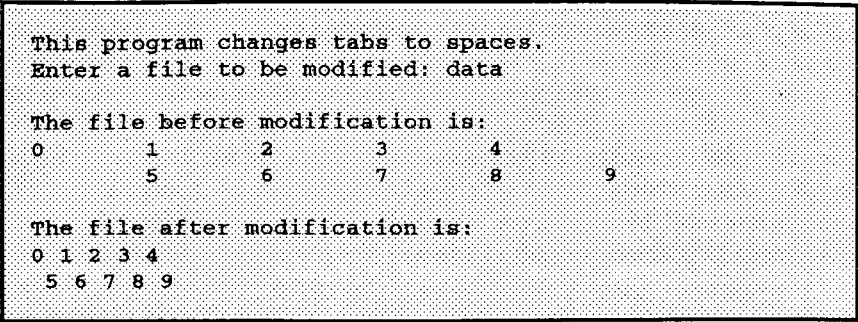


Fig. 14.6 Cómo utilizar los archivos temporales (parte 2 de 2).

14.10 Manejo de señales

Un evento inesperado o *señal*, puede hacer que termine un programa en forma prematura. Algunos eventos inesperados son *interrupciones* (escribir `<ctrl> c` en un sistema UNIX o en DOS), *instrucciones ilegales*, *violaciones de segmentación*, *órdenes de terminación provenientes del sistema operativo* y *excepciones de punto flotante* (división entre cero o la multiplicación de valores de punto flotante grandes). Mediante la función `signal`, la biblioteca de manejo de señales da la capacidad de *atrapar* eventos inesperados. La función `signal` recibe dos argumentos un número de señal entero y un apuntador a la función de manejo de señal. Las señales pueden ser generadas por la función `raise` que toma como argumento un número de señal entero. En la figura 14.7 se resumen las señales estándar, definidas en el archivo de cabecera `signal.h`. El programa de la figura 14.8 demuestra el uso de las funciones `signal` y `raise`.

El programa de la figura 14.8 utiliza la función `signal` para atrapar una señal interactiva (`SIGINT`). El programa empieza llamando `signal` mediante `SIGINT` y un apuntador a la función `signal_handler` (recuerde que el nombre de una función es un apuntador al principio de la misma). Cuando se genera una señal del tipo `SIGINT`, se pasa el control a la función `signal_handler`, se imprime un mensaje y se le da al usuario la opción de continuar con la

Señal	Explicación
SIGABRT	Terminación anormal del programa (como una llamada a la función <code>abort</code>).
SIGFPE	Una operación aritmética errónea, como sería una división entre cero o una operación que resulte en un desbordamiento.
SIGILL	Detección de una instrucción ilegal.
SIGINT	Recepción de una señal de atención interactiva.
SIGSEGV	Un acceso inválido a almacenamiento.
SIGTERM	Una solicitud de terminación definida al programa.

Fig. 14.7 Las señales definidas en el encabezado `signal.h`.

ejecución normal del programa. Si el usuario desea continuar con la ejecución, el manejador de señal es reinicializado, llamando otra vez a **signal** (algunos sistemas requieren que el manejador de señal sea reinicializado), y el control regresa al punto en el programa donde la señal fue detectada. En este programa, se utiliza la función **raise** para simular una señal interactiva. Se escoge un número al azar entre 1 y 50. Si el número es 25, entonces se llama a **raise** para generar la señal. Por lo regular, la señales interactivas son iniciadas desde fuera del programa. Por ejemplo, si en un sistema UNIX o DOS se escribe `<ctrl> c` durante la ejecución del programa, se genera una señal interactiva, que termina la ejecución del programa. El manejo de señales puede ser utilizado para atrapar una señal interactiva y evitar que el programa se dé por terminado.

14.11 Asignación dinámica de memoria: funciones **Calloc** y **Realloc**

En el capítulo 12, "Estructuras de datos", se presentó la noción de la asignación dinámica de la memoria, mediante el uso de la función **malloc**. Como se explicó en el capítulo 12, tratándose de ordenamiento rápido, búsqueda y acceso a los datos los arreglos resultan mejores que las listas enlazadas. Sin embargo, por lo regular los arreglos son *estructuras estáticas de datos*. Para la asignación dinámica de memoria, la biblioteca general de utilerías (**stdlib.h**) contiene otras dos funciones —**calloc** y **realloc**. Estas funciones pueden ser utilizadas para crear y modificar *arreglos dinámicos*. Como fue mostrado en el capítulo 7, "Apuntadores", un apuntador a un arreglo puede ser marcado con un subíndice, de la misma forma que un arreglo. Entonces, un apuntador a una porción contigua de memoria generada por **calloc**, puede ser manipulado como si fuera un arreglo. La función **calloc** asigna de forma dinámica la memoria para un arreglo. El prototipo para **calloc** es

```
void *calloc(size_t nmemb, size_t size);
```

Recibe dos argumentos —el número de elementos (**nmemb**) y el tamaño de cada elemento (**size**)— e inicializa los elementos del arreglo a cero. La función regresa un apuntador a la memoria asignada, o un apuntador **NULL** si no se asigna memoria.

La función **realloc** modifica el tamaño de un objeto asignado mediante una llamada a **malloc**, **calloc** o **realloc** anterior. Siempre y cuando la cantidad de memoria asignada sea mayor que la cantidad ya asignada, el contenido del objeto original no es modificado. De lo contrario, se conservará el contenido sin modificación hasta el tamaño del nuevo objeto. El prototipo correspondiente a **realloc** es

```
void *realloc(void *ptr, size_t size);
```

La función **realloc** toma dos argumentos un apuntador al objeto original (**ptr**) y el nuevo tamaño del objeto (**size**). Si **ptr** es **NUL**, **realloc** funciona en forma idéntica a **malloc**. Si **size** es 0 y **ptr** no es **NULL**, se libera la memoria correspondiente al objeto. De lo contrario, si **ptr** no es **NULL** y el tamaño es mayor que 0, **realloc** intenta asignar un nuevo bloque de memoria para el objeto. Si el nuevo espacio no puede ser asignado, el objeto al cual señala **ptr** se conserva sin modificar. La función **realloc** regresa ya sea un apuntador a la memoria reasignada, o un apuntador **NULL**.

14.12 La bifurcación incondicional: **Goto**

A lo largo del texto hemos insistido en la importancia de utilizar técnicas de programación estructurada para construir software confiable, que resulte fácil de depurar, mantener y modificar. En algunos casos, el rendimiento es de mayor importancia que una estricta adherencia a las

```
/* Using signal handling */

#include <stdio.h>
#include <signal.h>
#include <stdlib.h>
#include <time.h>

void signal_handler(int);

main()
{
    int i, x;

    signal(SIGINT, signal_handler);
    srand(clock());

    for (i = 1; i <= 100; i++) {
        x = 1 + rand() % 50;

        if (x == 25)
            raise(SIGINT);

        printf("%4d", i);

        if (i % 10 == 0)
            printf("\n");
    }

    return 0;
}

void signal_handler(int signalValue)
{
    int response;

    printf("%s%d%s\n%s",
        "\nInterrupt signal (", signalValue, ") received.",
        "Do you wish to continue (1 = yes or 2 = no)? ");

    scanf("%d", &response);

    while (response != 1 && response != 2) {
        printf("(1 = yes or 2 = no)? ");
        scanf("%d", &response);
    }

    if (response == 1)
        signal(SIGINT, signal_handler);
    else
        exit(EXIT_SUCCESS);
}
```

Fig. 14.8 Cómo utilizar el manejo de señales (parte 1 de 2).

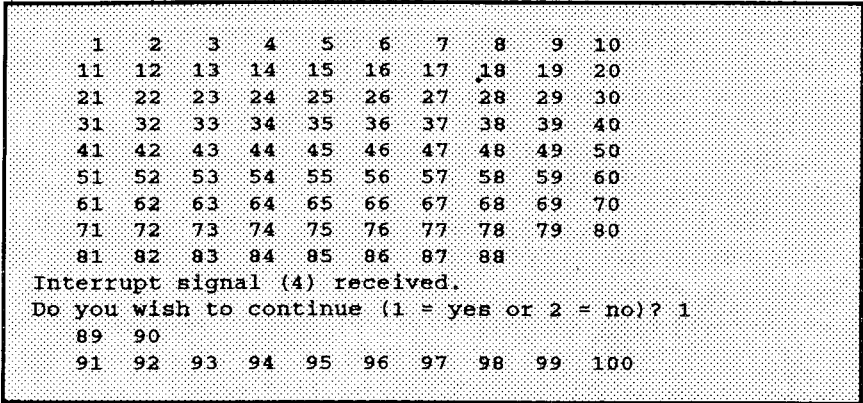


Fig. 14.8 Cómo utilizar el manejo de señales (parte 2 de 2).

técnicas de programación estructurada. En estos casos, pueden utilizarse algunas técnicas de programación no estructuradas. Por ejemplo, podemos utilizar **break** para terminar la ejecución de una estructura de repetición, antes de que se convierta en falsa la condición de continuación de ciclo. Esto ahorrará repeticiones innecesarias del ciclo, si antes de la terminación de éste la tarea ha sido terminada.

Otro ejemplo de programación no estructurada es el *enunciado goto* —una bifurcación incondicional. El resultado del enunciado **goto** es un cambio en el flujo de control del programa al primer enunciado que siga a la *etiqueta* especificada en el enunciado **goto**. Una etiqueta es un identificador seguido por dos puntos. Una etiqueta deberá aparecer en la misma función que el enunciado **goto** al cual se hace referencia. El programa de la figura 14.9 utiliza enunciados **goto** para ciclar diez veces e imprimir el valor de contador cada una de las veces. Después de inicializar **count** a 1, el programa verifica **count** para determinar si es mayor que 10 (la etiqueta **start** es pasada por alto, porque las etiquetas no ejecutan ninguna acción). De ser así, el control se transfiere desde **goto** al primer enunciado después de la etiqueta **end**. De lo contrario, se imprime **count** y se incrementa, y el control se transfiere de **goto** al primer enunciado después de la etiqueta **start**.

En el capítulo 3 indicamos que para escribir cualquier programa sólo eran requeridas 3 estructuras de control: secuencia, selección y repetición. Cuando se siguen las reglas de la programación estructurada, es posible crear estructuras de control muy anidadas, de las cuales es difícil salir con eficacia. En situaciones como esas, algunos programadores utilizan enunciados **goto**, como una salida rápida de una estructura muy anidada. Esto elimina la necesidad de probar múltiples condiciones para salir de una estructura de control.

Sugerencia de rendimiento 14.3

El enunciado **goto** puede ser utilizado para salir con eficacia de estructuras de control muy anidadas.

Observación de ingeniería de software 14.3

El enunciado **goto** debe ser utilizado sólo en aplicaciones orientadas a rendimiento. El enunciado **goto** no es estructurado y puede llevar a programas más difíciles de depurar, mantener y modificar.

```
/* Using goto */
#include <stdio.h>

main()
{
    int count = 1;

    start:                                /* label */
        if (count > 10)
            goto end;

        printf("%d ", count);
        ++count;
        goto start;

    end:                                  /* label */
        putchar('\n');

    return 0;
}
```



Fig. 14.9 Cómo utilizar goto.

Resumen

- En muchos sistemas de cómputo y en particular, en sistemas UNIX y DOS es posible redirigir la entrada a un programa y redirigir la salida de un programa.
- La entrada se redirige desde la línea de comandos de UNIX y de DOS utilizando el símbolo de redirección de entrada (<) o mediante una tubería (|).
- La salida es redirigida desde la línea de comandos de UNIX y de DOS utilizando el símbolo de redirección de salida (>) o el símbolo de agregar salida (>>). El símbolo de redirección de salida sólo almacena la salida de programa en un archivo, y el símbolo de agregar salida, agrega la salida al final de un archivo.
- Las macros y definiciones del encabezado de argumentos variables **stdarg.h** proporcionan las capacidades necesarias para construir funciones utilizando listas de argumentos de longitud variable.
- Unos puntos suspensivos (...) en una función prototipo indican un número variable de argumentos.
- El tipo **va_list** es adecuado para contener información necesaria para las macros **va_start**, **va_arg**, y **va_end**. Para tener acceso a los argumentos en una lista de argumentos de longitud variable, deberá ser declarado un objeto del tipo **va_list**.
- Antes de poder tener acceso a los argumentos de una lista de argumentos de longitud variable, se invoca la macro **va_start**. La macro inicializa el objeto declarado mediante **va_list**, para su uso con las macros **va_arg**, y **va_end**.

- La macro **va_arg** se expande a una expresión del valor y del tipo del siguiente argumento de la lista de argumentos de longitud variable. Cada invocación de **va_arg** modifica el objeto declarado en **va_list**, de tal forma que el objeto señale al siguiente argumento dentro de la lista.
- La macro **va_end** facilita un regreso normal de una función cuya lista de argumentos variables fue referida por la macro **va_start**.
- En muchos sistemas y en particular en DOS y en UNIX incluyendo los parámetros **int argc** y **char *argv[]** en la lista de parámetros de **main**, es posible pasar argumentos a **main** desde la línea de comandos. El parámetro **argc** recibe el número de argumentos de la línea de comandos. El parámetro **argv** es un arreglo de cadenas, en el cual se almacenan los argumentos actuales de la línea de comandos.
- La definición de una función debe estar en su totalidad contenida en un archivo, no puede estar distribuida en dos o más archivos.
- Las variables globales deberán ser declaradas en cada uno de los archivos en las que serán utilizadas.
- Los prototipos de función pueden extender el alcance de una función más allá del archivo en el cual ha sido definida (en un prototipo de función no es requerido el especificador **extern**). Esto se lleva a cabo incluyendo a el prototipo de función en cada archivo en el cual sea invocada la función y compilando juntos ambos archivos.
- El especificador de clase de almacenamiento **static**, al ser aplicado a la variable global o a una función, impide que ésta sea utilizada por cualquier función que no haya sido definida en el mismo archivo. Esto se conoce como enlace interno. Las variables globales y las funciones que en sus definiciones no hayan sido precedidas por **static** tienen enlace externo —se puede tener acceso a ellas en otros archivos, si estos archivos contienen las declaraciones y/o los prototipos de función adecuadas.
- El especificador **static** se usa por lo general con funciones de utilidad, que son llamadas sólo por funciones en un archivo en particular. Si una función no es requerida fuera de un archivo particular, deberá ser puesto en aplicación el principio del mínimo privilegio mediante el uso de **static**.
- Al construir programas extensos en múltiples archivos fuente, la compilación de programas se hace tediosa si al efectuar pequeñas modificaciones a un archivo se tiene que recompilar todo el programa. Muchos sistemas proporcionan utilerías especiales que recompila sólo el archivo de programa modificado. En sistemas UNIX la utilería se llama **make**. La utilería **make** lee un archivo llamado **makefile** que tiene instrucciones para compilar y enlazar el programa.
- La función **exit** obliga a la terminación de un programa como si se hubiera ejecutado normalmente.
- La función **atexit** registra una función en un programa, para que sea llamada a la terminación del programa es decir, ya sea cuando el programa termina por llegar al final de **main** o cuando se invoca a **exit**.
- La función **atexit** toma como argumento un apuntador a una función (es decir, el nombre de la función). Las funciones llamadas a la terminación del programa no pueden tener argumentos, y no pueden regresar un valor. Pueden registrarse hasta 32 funciones para su ejecución a la terminación del programa.

- La función **exit** toma un argumento. Por lo regular el argumento es la constante simbólica **EXIT_SUCCESS** o la constante simbólica **EXIT_FAILURE**. Si **exit** se llama con **EXIT_SUCCESS**, es regresado el valor definido por la puesta en práctica correspondiente a la terminación exitosa al entorno llamador. Si **exit** es llamado mediante **EXIT_FAILURE** el valor definido por la puesta en práctica correspondiente a la terminación no exitosa, es regresado.
- Cuando se invoca la función **exit**, cualesquiera funciones registradas con **atexit** son invocadas en orden inverso a su registro, todos los flujos asociados con el programa son vaciados y cerrados, y el control regresa al entorno huésped.
- El estándar ANSI (An90) indica que cuando se utiliza **volatile** para calificar un tipo, la naturaleza del acceso a un objeto de dicho tipo depende de la puesta en práctica. Kernighan y Ritchie (Ke88) indican que el calificador **volatile** se utiliza para suprimir varios tipos de optimizaciones.
- C proporciona sufijos enteros y de punto flotante para especificar los tipos de constantes enteras y de punto flotante. Los sufijos enteros son: **u** o bien **U** por un entero **unsigned**, **l** o **L** por un entero **long**, y **ul** o **UL** por un entero **unsigned long**. Si una constante entera no tiene sufijo, su tipo queda determinado por el primer tipo capaz de almacenar un valor de dicho tamaño (primero **int**, luego **long int** y por último **unsigned long int**). Los sufijos de punto flotante son: **f** o **F** para un **float**, y **l** o **L** para un **long double**. Una constante de punto flotante sin sufijo es del tipo **double**.
- C también proporciona capacidades para procesamiento de archivos binarios, pero algunos sistemas de cómputo no aceptan archivos binarios. Si los archivos binarios no son aceptados, y un archivo se abre en el modo de archivo binario, el archivo será procesado como archivo de texto.
- La función **tempfile** abre un archivo temporal en modo "**wb+**". A pesar de que éste es un modo de archivo binario, algunos sistemas procesan los archivos temporales como archivos de texto. Un archivo temporal existe en tanto no sea cerrado mediante **fclose** o en tanto termine el programa.
- La biblioteca de manejo de señales proporciona la capacidad de atrapar eventos inesperados mediante la función **signal**. La función **signal** recibe dos argumentos un número de señal entero y un apuntador a la función de manejo de señal.
- Las señales también pueden ser generadas mediante la función **raise** y un argumento entero.
- La biblioteca general de utilerías (**stdlib.h**) tiene dos funciones para la asignación dinámica de memoria **calloc** y **realloc**. Estas funciones pueden ser utilizadas para crear arreglos dinámicos.
- La función **calloc** recibe dos argumentos el número de elementos (**nmemb**) y el tamaño de cada elemento (**size**), e inicializa a cero los elementos del arreglo. La función regresa ya sea un apuntador a la memoria asignada, o un apuntador **NULL** si la memoria no ha sido asignada.
- La función **realloc** modifica el tamaño de un objeto asignado por una llamada a **malloc**, **calloc** o **realloc** previa. El contenido del objeto original no es modificado, siempre y cuando la cantidad de memoria asignada sea mayor que la cantidad ya asignada.
- La función **realloc** toma dos argumentos un apuntador al objeto original (**ptr**) y el nuevo tamaño del objeto (**size**). Si **ptr** es **NULL**, **realloc** funciona en forma idéntica a **malloc**. Si **size** es 0 y el apuntador recibido no es **NULL**, la memoria para el objeto es liberada. De lo contrario si **ptr** no es **NULL** y el tamaño es mayor que 0, **realloc** intenta asignar un nuevo

contrario si `ptr` no es `NULL` y el tamaño es mayor que 0, `realloc` intenta asignar un nuevo bloque de memoria para el objeto. Si el nuevo espacio no puede ser asignado, el objeto al cual señala `ptr`, se conserva sin modificar. La función `realloc` regresa ya sea un apuntador a la memoria reasignada, o un apuntador `NULL`.

- El resultado del enunciado `goto` es una modificación en el flujo de control del programa. La ejecución del programa continúa en el primer enunciado inmediatamente después de la etiqueta especificada en el enunciado `goto`.
- Una etiqueta es un identificador seguido por dos puntos. Debe de aparecer una etiqueta en la misma función que en el enunciado `goto` que se refiere a ella.

Terminología

símbolo de agregar salida >>	<code>makefile</code>
<code>argc</code>	tubería
<code>argv</code>	entubamiento
<code>atexit</code>	<code>raise</code>
<code>calloc</code>	<code>realloc</code>
argumentos en la línea de comandos	símbolo de redirección de entrada <
<code>const</code>	símbolo de redirección de salida >
arreglos dinámicos	violación de segmentación
evento	<code>signal</code>
<code>exit</code>	biblioteca de manejo de señales
enlace externo	<code>signal.h</code>
especificador de clase de almacenamiento <code>extern</code>	especificador de clase de almacenamiento <code>static</code>
<code>EXIT_FAILURE</code>	<code>stdarg.h</code>
<code>EXIT_SUCCESS</code>	archivo temporal
sufijo <code>float</code> (f o F)	<code>tmpfile</code>
excepción de punto flotante	atrapar
enunciado <code>goto</code>	sufijo entero <code>unsigned</code> (u o bien U)
redirección de E/S	sufijo entero <code>unsigned long</code> (ul o bien UL)
instrucción ilegal	<code>va_arg</code>
enlace interno	<code>va_end</code>
interrupción	<code>va_list</code>
sufijo <code>long double</code> (l o L)	<code>va_start</code>
sufijo <code>long integer</code> (l o L)	lista de argumentos de longitud variable
<code>make</code>	<code>volatile</code>

Error común de programación

- 14.1 Colocar puntos suspensivos en la mitad de una lista de parámetros de función. Los puntos suspensivos sólo pueden ser colocados al final de la lista de parámetros.

Sugerencias de portabilidad

- 14.1 Algunos sistemas no aceptan nombres de variables globales o nombres de funciones con más de 6 caracteres. Esto deberá ser tomado en cuenta al escribir programas que se planea serán transportados a varias plataformas.
- 14.2 Cuando esté escribiendo programas portátiles utilice archivos de texto.

Sugerencias de rendimiento

- 14.1 Las variables globales aumentan el rendimiento debido a que son de forma directa accesibles por cualquier función —se elimina la sobrecarga de pasar datos a función.
- 14.2 Considere utilizar archivos binarios en lugar de archivos de texto en aquellas aplicaciones que exijan alto rendimiento
- 14.3 El enunciado `goto` puede ser utilizado para salir con eficacia de estructuras de control muy anidadas.

Observaciones de ingeniería de software

- 14.1 Las variables globales deberían de evitarse, a menos de que el rendimiento de la aplicación sea crítico, porque violan el principio del mínimo privilegio.
- 14.2 Las funciones pueden ser comunes para muchas aplicaciones. En casos como éstos, estas funciones deberán ser almacenadas en sus propios archivos fuente, y cada archivo fuente deberá tener su correspondiente archivo de cabecera, que contenga los prototipos de función. Esto permite a los programadores de distintas aplicaciones reutilizar el mismo código, mediante la inclusión del archivo de cabecera apropiado, y compilar su aplicación con el archivo fuente correspondiente.
- 14.3 El enunciado `goto` debe ser utilizado sólo en aplicaciones orientadas a rendimiento. El enunciado `goto` no es estructurado y puede llevar a programas más difíciles de depurar, mantener y modificar.

Ejercicios de autoevaluación

- 14.1 Llene los espacios vacíos de cada uno de los siguientes:
- a) El símbolo _____ se utiliza para redirigir datos de entrada del teclado para que provengan de un archivo.
 - b) El símbolo _____ se utiliza para redirigir la salida a pantalla para que se coloque en un archivo.
 - c) El símbolo _____ se utiliza para agregar la salida de un programa al final de un archivo.
 - d) La _____ se utiliza para redirigir la salida de un programa como entrada de otro programa.
 - e) Una _____ en la lista de parámetros de una función indica que la función puede recibir un número variable de argumentos.
 - f) La macro _____ debe de ser invocada antes de que se pueda tener acceso a los argumentos de una lista de argumentos de longitud variable.
 - g) Se utiliza la macro _____ para tener acceso a los argumentos individuales de una lista de argumentos de longitud variable.
 - h) La macro _____ facilita un regreso normal de una función cuya lista de argumentos variable fue referida por la macro `va_start`.
 - i) El argumento _____ de `main` recibe el número de argumentos en la línea de comandos.
 - j) El argumento _____ de `main` almacena argumentos en la línea de comandos como cadenas de caracteres.
 - k) La utilidad de UNIX _____ lee un archivo llamado _____ que contiene instrucciones para compilar y enlazar un programa formado de múltiples archivos fuente. La utilidad también recompila el archivo si desde la última vez que fue compilado éste ha sido modificado.
 - l) La función _____ obliga a un programa a terminar su ejecución.
 - m) La función _____ registra una función que deberá ser llamada a la terminación normal del programa.
 - n) El calificador de tipo _____ indica que un objeto no deberá ser modificado después de haberse inicializado.
 - o) Se puede agregar un _____ entero o de punto flotante a una constante entera o de punto flotante para especificar el tipo exacto de la misma.

- p) La función _____ abre un archivo temporal que existirá en tanto no se cierre o la ejecución del programa se termine.
- q) La función _____ puede ser utilizada para atrapar eventos inesperados.
- r) La función _____ genera una señal desde dentro de un programa.
- s) La función _____ asigna dinámicamente la memoria para un arreglo que inicializa los elementos a cero.
- t) La función _____ modifica el tamaño de un bloque de memoria ya asignada dinámicamente.

Respuestas a los ejercicios de autoevaluación

14.1 a) redirige entrada (<). b) redirige salida (>). c) agrega salida (>>). d) tubería (|). e) puntos suspensivos (...). f) `va_start`. g) `va_arg`. h) `va_end`. i) `argc`. j) `argv`. k) `make`, `makefile`. l) `exit`. m) `atexit`. n) `const`. o) sufijo. p) `tempfile`. q) `signal`. r) `raise`. s) `calloc`. t) `realloc`.

Ejercicios

14.2 Escriba un programa que calcule el producto de una serie de enteros que son pasados a la función `product` utilizando una lista de argumentos de longitud variable. Pruebe su función con varias llamadas, cada una con un número diferente de argumentos.

14.3 Escriba un programa que imprima los argumentos en la línea de comandos del programa.

14.4 Escriba un programa que ordene un arreglo de enteros en orden ascendente o en orden descendente. El programa deberá utilizar argumentos en la línea de comandos, para pasar o el argumento `-a` para orden ascendente o el argumento `-d` para orden descendente. (Nota: este es el formato estándar en UNIX para pasar opciones a un programa.)

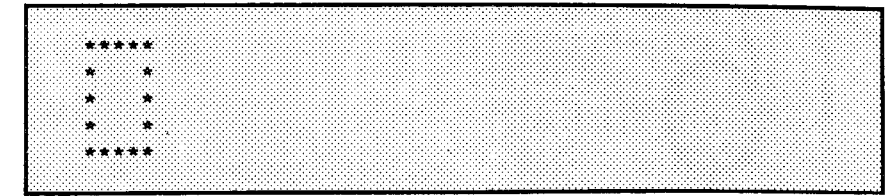
14.5 Escriba un programa que coloque un espacio entre cada carácter dentro de un archivo. El programa deberá primero escribir el contenido de un archivo a modificarse a un archivo temporal con espacios entre cada carácter, y a continuación copiar el archivo de regreso al archivo original. Esta operación deberá sobrescribir el contenido original del archivo.

14.6 Lea los manuales de su sistema para determinar qué señales contienen la biblioteca de manejo de señales (`signal.h`). Escriba un programa que contenga los manejadores de señales correspondientes a las señales estándar `SIGABRT` y `SIGINT`. El programa deberá probar el método de atrapar de estas señales mediante el llamado a las funciones `abort` para generar una señal del tipo `SIGABRT`, y mediante la escritura de `<ctrl> c` para generar una señal del tipo `SIGINT`.

14.7 Escriba un programa que asigne de forma dinámica un arreglo de enteros. El tamaño del arreglo deberá ser introducido desde el teclado. Los elementos del arreglo deberán ser valores asignados introducidos desde el teclado. Imprima los valores del arreglo. A continuación, reasigne la memoria para el arreglo a la mitad del número actual de elementos. Imprima los valores restantes en el arreglo para confirmar que corresponden a la primera mitad de los valores del arreglo original.

14.8 Escriba un programa que tome dos argumentos de la línea de comandos que son nombres de archivo, lea los caracteres del primer archivo un carácter a la vez, y escriba los caracteres en orden inverso en el segundo archivo.

14.9 Escriba un programa que utilice enunciados `goto` para simular una estructura de ciclado anidada, que imprima un cuadro de asteriscos como sigue:



El programa deberá utilizar sólo los siguientes tres enunciados `printf`:

```
printf("**");
printf(" ");
printf("\n");
```

15

C++ como un “C mejorado”

Objetivos

- Familiarizarse con las mejoras de C++ a C.
- Reconocer la razón porqué C es una base para subsecuentes estudios de la programación en general y de C ++ en particular.

*Lo mejor es verte
con mi estimado.*

El gran lobo feroz a la pequeña caperucita roja

Sinopsis

- 15.1 Introducción
- 15.2 Comentarios en una sola línea de C++
- 15.3 Flujo de entrada/salida de C++
- 15.4 Declaraciones en C++
- 15.5 Cómo crear nuevos tipos de datos en C++
- 15.6 Prototipos de función y verificación de tipo
- 15.7 Funciones en línea (inline)
- 15.8 Parámetros de referencia
- 15.9 El calificador Const
- 15.10 Asignación dinámica de memoria mediante new y delete
- 15.11 Argumentos por omisión
- 15.12 Operador de resolución de alcance unario
- 15.13 Homonimia de función
- 15.14 Especificaciones de enlace
- 15.15 Plantillas de función

Resumen • Terminología • Errores comunes de programación • Prácticas sanas de programación • Sugerencia de portabilidad • Sugerencias de rendimiento • Observación de ingeniería de software • Ejercicios de autoevaluación • Respuestas a los ejercicios de autoevaluación • Ejercicios • Lectura recomendada • Apéndice: recursos de C++.

15.1 Introducción

¡Bienvenido a C++! C++ es una mejora sobre muchas de las características de C, y proporciona capacidades de programación orientada a objetos (OOP, por *object-oriented programming*) que promete mucho para incrementar la productividad, calidad y reutilización del software. El resto de este capítulo analiza muchas de las mejoras que C++ tiene sobre C.

Los diseñadores de C y los responsables de sus primeras puestas en práctica jamás anticiparon que este lenguaje resultaría en un fenómeno como éste (lo mismo es cierto para el sistema operativo UNIX). Cuando un lenguaje de programación se torna tan arraigado como C, nuevas necesidades demandan que el lenguaje evolucione, en lugar de que sólo sea remplazado por un nuevo lenguaje.

C++ fue desarrollado por Bjarne Stroustrup en los Laboratorios Bell (St86) y originalmente fue llamado "C con clases". El nombre C++ incluye el operador de incremento (++) de C, para indicar que C++ es una versión mejorada de C.

C++ es un super conjunto de C, por lo que, para compilar los programas existentes de C, los programadores pueden utilizar un compilador C++ y posteriormente modificar de forma gradual estos programas a C++. En este momento, ya algunos proveedores importantes de software ofrecen compiladores C++ y no ofrecen productos C por separado.

Aún no existe un estándar ANSI, a pesar de que un comité ANSI está desarrollando una proposición. Muchas personas creen que para mediados de los años noventa, la mayor parte de los entornos de programación C se convertirán a C++.

15.2 Comentarios de una sola línea de C++

Con frecuencia los programadores insertan un pequeño comentario al final de una línea de código. C requiere que un comentario sea delimitado mediante /* y */. C++ le permitirá a que empiece un comentario con // y que utilice el resto de la línea para texto del comentario; el fin de la línea da de manera automática por terminado el comentario. Esta forma de comentario ahorra algunos golpes de tecla, pero sólo es aplicable a comentarios que no continúen a la línea siguiente.

Por ejemplo, el comentario C

```
/* This is a single-line comment. */
```

requiere de ambos delimitadores /* y */, aun para un comentario de una línea. La versión en una línea de C++ de lo anterior es

```
// This is a single-line comment.
```

Para comentarios de más de una línea, como

```
/* This is one way to
   * Write neat multiple-
   * line comments.      */
```

La notación C++ pudiera aparecer más concisa como en

```
// This is one way to
// Write neat multiple-
// line comments.
```

Pero recuerde que los comentarios C pueden extenderse a varias líneas, por lo que en realidad, sólo se necesita un par (en vez de que de los tres pares que hemos utilizado) de delimitadores de comentario, como en

```
/* This is one way to
   * Write neat multiple-
   * line comments.      */
```

Dado que C++ es un super conjunto de C, en un programa C++ ambas formas de comentario son aceptables.

Error común de programación 15.1

*Olvidar cerrar un comentario de estilo C mediante */.*

Práctica sana de programación 15.1

*Usar los comentarios // de estilo C++, evita errores de comentarios sin terminar, que ocurren al olvidarse de cerrar los comentarios de estilo C con */.*

Este error en apariencia sencillo puede llevar a errores muy sutiles, algunos de los cuales pueden deslizarse sin detección por parte del compilador. El efecto de omitir el `*/` es que el compilador supone que el comentario aún no ha terminado, y que el comentario continúa a lo largo de las líneas subsiguientes hasta que alcanza otro `*/`, correspondiente a otro comentario, o el final del archivo del programa. Esto podría dar como resultado que el compilador ignore alguna parte clave del programa.

15.3 Flujo de entrada/salida de C++

C++ ofrece una alternativa a las llamadas de función `printf` y `scanf` para manejar la entrada/salida de los tipos y cadenas de datos estándar. Por ejemplo, el diálogo simple

```
printf("Enter new tag: ");
scanf("%d", &tag);
printf("The new tag is: %d\n", tag);
```

se escribe en C++ como

```
cout << "Enter new tag: ";
cin >> Tag;
cout << "The new tag is: " << tag << '\n';
```

El primer enunciado utiliza el *flujo estándar de salida* `cout` y el operador `<<` (el *operador de inserción de flujo* que se pronuncia "colocar en"). El enunciado se lee

La cadena "Enter new tag" es colocada en el flujo de salida cout.

Note que el operador de inserción de flujo es también el operador de desplazamiento a la izquierda a nivel de bits. El segundo enunciado utiliza el *flujo de entrada estándar* `cin` y el operador `>>` (el *operador de extracción de flujo*, que se pronuncia "obtener de"). El enunciado se lee

Obtener un valor para tag del flujo de entrada cin.

Note que el operador de extracción de flujo también es un operador de desplazamiento a la derecha a nivel de bits cuando el argumento izquierdo es un tipo entero. Los operadores de inserción y de extracción de flujo, a diferencia de `printf` y de `scanf`, no requieren de cadenas de formato y de especificadores de conversión para indicar los tipos de datos que son extraídos o introducidos. C++ tiene muchos ejemplos como éste, en los cuales de forma automática "sabe" qué tipos utilizar. También note que, cuando se utiliza con el operador de extracción de flujo, la variable `tag` no es precedida por el operador de dirección `&`, como es requerido en el caso de `scanf`.

Para utilizar entradas/salidas de flujo, los programas C++ deben incluir el archivo de cabecera `iostream.h`. El programa de la figura 15.1 solicita las variables `myAge` y `friendsAge` y determina si usted es de más edad, menos edad o de la misma que su amigo. Note la colocación de las declaraciones de edad, justo antes de la referenciación de las edades en los enunciados de entrada. En el capítulo 19, "Flujo de entrada/salida" se da una descripción detallada de las características del flujo de entrada/salida de C++.

Práctica sana de programación 15.2

Utilizar entrada/salida orientada a flujo de tipo C++ hace los programas más legibles (y menos sujetos a errores), que sus contrapartidas escritas en C mediante las llamadas de función `printf` y `scanf`.

```
// Simple stream input/output
#include <iostream.h>

main()
{
    cout << "Enter your age: ";
    int myAge;
    cin >> myAge;

    cout << "Enter your friend's age: ";
    int friendsAge;
    cin >> friendsAge;

    if (myAge > friendsAge)
        cout << "You are older.\n";
    else
        if (myAge < friendsAge)
            cout << "You are younger.\n";
        else
            cout << "You and your friend are the same age.\n";

    return 0;
}
```

```
Enter your age: 23
Enter your friend's age: 20
You are older.
```

```
Enter your age: 20
Enter your friend's age: 23
You are younger.
```

```
Enter your age: 20
Enter your friend's age: 20
You and your friend are the same age.
```

Fig. 15.1 Flujo de E/S y los operadores de Inserción y extracción de flujo.

15.4 Declaraciones en C++

En un bloque en C, todas las declaraciones deben aparecer antes de cualquier enunciado ejecutable. En C++, las declaraciones pueden ser colocadas en cualquier parte de un enunciado ejecutable, siempre y cuando las declaraciones antecedan el uso de lo que se está declarando. Por ejemplo

```
cout << "Enter two integers: ";
int x, y;
cin >> x >> y;
cout << "The sum of " << x << " and " << y
    << " is " << x + y << '\n';
```

declara las variables **x** y **y** después del enunciado ejecutable **cout**, pero antes que sean utilizadas en el enunciado subsecuente **cin**. También, las variables pueden ser declaradas en la sección de inicialización de una estructura **for** dichas variables se mantienen en alcance hasta el final del bloque en el cual la estructura **for** está definida. Por ejemplo,

```
for (int i = 0; i <= 5; i++)
    count << i << '\n';
```

declara la variable **i** ser un entero y la inicializa a 0 en la estructura **for**.

El alcance de una variable local C++ empieza en su declaración y se extiende hasta la llave derecha de cierre (**}**). Por lo tanto, los enunciados anteriores a la declaración variable no pueden referirse a la variable, aun si éstos están en el mismo bloque. Las declaraciones de variable no pueden ser colocadas en la condición de una estructura **while**, **do/while**, **for**, o **if**.

Práctica sana de programación 15.3

Colocar la declaración de una variable cerca de su primer uso puede hacer los programas más legibles.

Error común de programación 15.2

Declarar una variable después de que haya sido referenciada en un enunciado.

15.5 Cómo crear nuevos tipos de datos en C++

C++ proporciona la capacidad de crear tipos definidos por el usuario mediante el uso de la palabra reservada **enum**, la palabra reservada **struct**, la palabra reservada **union** y la nueva palabra reservada **class**. Al igual que en C, las enumeraciones C++ son declaradas mediante **enum**. Sin embargo; a diferencia de C, una enumeración en C++, cuando se declara, se convierte en un tipo nuevo. Para declarar de variable del nuevo tipo la palabra reservada **enum** no es requerida. Lo mismo es cierto en el caso de **struct**, **union** y **class**. Por ejemplo, las declaraciones

```
enum Boolean {FALSE, TRUE};

struct Name {
    char first[10];
    char last[10];
};
union Number {
    int i;
    float f;
};
```

crean tres tipos de datos, definidos por usuario, con los nombres de etiqueta **Boolean**, **Name** y **Number**. Los nombres de etiqueta pueden ser utilizados para declarar variables, como sigue:

```
Boolean done = FALSE;
Name student;
Number x;
```

Estas declaraciones crean la variable **Boolean done** (inicializada a **FALSE**), la variable **Name student** y la variable **Number x**.

Como en C, las enumeraciones por omisión son valuadas iniciándose en cero y cada elemento subsecuente se incrementa en uno. Por lo tanto, la enumeración **Boolean** asigna el valor 0 a **FALSE** y 1 a **TRUE**. Cualquier elemento en una enumeración puede ser asignado un valor entero. Los elementos subsecuentes, que no sean asignados a un valor, recibirán de manera automática el valor del elemento anterior, incrementado en uno.

15.6 Prototipos de función y verificación de tipo

El prototipo de función le permiten al compilador de C verificar por tipo la exactitud de las llamadas de función. En ANSI C, los prototipos de función son opcionales. En C++ los prototipos de función son requeridas para todas las funciones. Una función definida en un archivo, antes de cualquier llamada a la misma, no requiere de un prototipo de función por separado. En este caso, el encabezado de función actúa como prototipo de función. C++ también requiere que se declaren todos los parámetros de función en los paréntesis de la definición de función y del prototipo. Por ejemplo, una función **square**, que toma un entero de argumento y regresa un entero tiene el prototipo:

```
int square(int);
```

Las funciones que no regresan un valor se declaran con el tipo de regreso **void**.

Error común de programación 15.3

Un intento de regresar un valor de una función **void** o de utilizar el resultado de una llamada a una función **void**.

Para especificar en C una lista vacía de parámetros, entre paréntesis se coloca la palabra reservada **void**. Si no se coloca nada en los paréntesis de un prototipo de función de C, para dicha función se desactiva toda verificación de argumentos y nada se supone en relación con el número de argumentos y los tipos de argumentos a la función. Cualquier llamada de función correspondiente a esta función puede pasar cualesquiera argumentos que desee, sin que el compilador reporte error alguno.

En C++, una lista vacía de parámetros se especifica escribiendo **void** o absolutamente nada en los paréntesis. La declaración

```
void print();
```

especifica que la función **print** no toma argumentos y no regresa un valor. En la figura 15.2 se muestran ambas formas de declarar en C++ y de usar las funciones que no toman argumentos.

Sugerencia de portabilidad 15.1

El significado de una lista de función de parámetros vacía es muy distinto en C++ que en C. En C, significa que se deshabilita toda verificación de argumentos. En C++, significa que la función no toma argumentos. Por lo tanto, si se compilan en C++, los programas en C que utilizan esta característica pudieran ejecutarse en forma diferente.

Error común de programación 15.4

Los programas en C++ no se compilarán a menos de que sean proporcionadas los prototipos de función para cada una de las funciones o que cada función quede definida antes de ser utilizada.

```
// Functions that take no arguments
#include <iostream.h>

void f1();
void f2(void);

main()
{
    f1();
    f2();

    return 0;
}

void f1()
{
    cout << "Function f1 takes no arguments\n";
}

void f2(void)
{
    cout << "Function f2 also takes no arguments\n";
}
```

```
Function f1 takes no arguments
Function f2 also takes no arguments
```

Fig. 15.2 Dos maneras de declarar y utilizar funciones que no toman argumentos.

15.7 Funciones en línea

Desde el punto de vista de la ingeniería del software es una buena idea poner en práctica un programa como un conjunto de funciones, pero las llamadas de función involucran sobrecarga en tiempo de ejecución. C++ tiene *funciones en línea* que ayudan a reducir la sobrecarga por llamadas de función especial para pequeñas funciones. El calificador `inline` colocado en la definición de función antes del tipo de regreso de una función "aconseja" al compilador que genere una copia del código de la función "in situ" (cuando sea apropiado), a fin de evitar una llamada de función. La contrapartida es que en el programa se insertan muchas copias de código de la función, en vez de, cada vez que se llama a la función, tener una copia de la función a la cual pasarle el control. El compilador puede ignorar el calificador `inline` y típicamente así lo hará para todas, a excepción de las funciones más pequeñas.

Observación de ingeniería del software 15.1

Cualquier modificación a una función `inline` requiere que sean recompilados todos los clientes de dicha función. Esto pudiera resultar de importancia en algunas situaciones de desarrollo y mantenimiento de programas.

Las funciones en línea ofrecen ventajas en comparación con las macros de preprocesador (vea el capítulo 13) que expanden código en línea. Una ventaja es que las funciones `inline` son iguales a cualquier otra función de C++. Por lo tanto, en llamadas a las funciones `inline` se ejecutará

una adecuada verificación de tipo; las macros de preprocesador no aceptan verificación de tipo. Otra ventaja es que las funciones `inline` eliminan los efectos colaterales inesperados, asociados con un uso inapropiado de las macros de preprocesador. Por último, las funciones `inline` pueden ser depuradas mediante un programa depurador. El depurador no reconoce a las macros de preprocesador como unidades especiales, porque sólo son sustituciones de texto, efectuadas por el preprocesador antes de la compilación del programa. Un depurador puede ayudar a localizar errores lógicos resultado de sustituciones de macros, pero no puede atribuir dichos errores a macros específicas.

Práctica sana de programación 15.4

El calificador `inline` deberá ser utilizado sólo tratándose de funciones pequeñas, de uso frecuente.

Sugerencia de rendimiento 15.1

Usar funciones `inline` puede reducir tiempo de ejecución, pero puede aumentar el tamaño del programa.

El programa de la figura 15.3 utiliza la función `inline` llamada `cube` para calcular el volumen de un cubo del lado `s`. La palabra reservada `const` en la lista de parámetros de la función `cube`, le indica al compilador que la función no modifica a la variable `s`. En la sección 15.9 se analiza con mayor detalle la palabra reservada `const`.

Las macros de preprocesador son operaciones definidas en una directiva de preprocesador `#define`. Una macro está compuesta de un nombre (identificador de la macro) y texto de remplazo. Las macros pueden ser definidas sin o con lista de argumentos. Las macros sin argumentos son procesadas como si fueran constantes simbólicas dentro del programa —el texto de remplazo sustituye el identificador de la macro. Las macros con argumentos sustituyen los

```
// Using an inline function to calculate
// the volume of a cube
#include <iostream.h>

inline float cube(const float s) { return s * s * s; }

main()
{
    cout << "Enter the side length of your cube: ";

    float side;

    cin >> side;
    cout << "Volume of cube with side "
        << side << " is " << cube(side) << '\n';

    return 0;
}
```

```
Enter the side length of your cube: 3.5
Volume of cube with side 3.5 is 42.875
```

Fig. 15.3 Cómo utilizar una función en línea para calcular el volumen de un cubo.

argumentos dentro del texto de remplazo y a continuación, expanden la macro dentro del programa.

Considere la siguiente macro de un argumento, para calcular la superficie de un cuadrado:

```
#define VALIDSQUARE(x) (x) * (x)
```

El preprocesador localiza dentro del programa cada ocurrencia de **VALIDSQUARE(x)**, sustituye **x** (ya sea un valor o una expresión) en el texto de remplazo, y expande la macro dentro del programa. Por ejemplo,

```
cout < VALIDSQUARE(7);
```

se expande a

```
cout < (7) * (7);
```

Los paréntesis en el texto de remplazo obligan a un orden correcto de evaluación de la operación de la macro. Por ejemplo,

```
cout << VALIDSQUARE(2 + 3);
```

se expande a

```
cout << (2 + 3) * (2 + 3);
```

que se evalúa correctamente como $5 * 5 = 25$. El preprocesador no evalúa expresiones incluidas como argumentos de una macro; sólo reemplaza cada instancia del nombre del argumento en el texto de remplazo, con una copia de la totalidad de la expresión. Por lo tanto, los paréntesis son necesarios, para asegurar que los argumentos son correctamente evaluados.

Considere una macro correspondiente, pero sin paréntesis en el texto de remplazo:

```
#define INVALIDSQUARE(x) x * x
```

Si damos $2 + 3$ como el argumento, la macro se expande como sigue:

```
2 + 3 * 2 + 3
```

Esta expresión queda evaluada de forma incorrecta como $2 + 6 + 3 = 11$ porque la multiplicación tiene una precedencia más alta que la suma.

C++ tiene las funciones en línea para eliminar los problemas asociados con las macros, para eliminar la sobrecarga asociada con las llamadas de función, y para asegurar la verificación de argumentos para las funciones. Los argumentos de las funciones en línea se evalúan antes de ser "pasados" a la función. Por lo tanto los paréntesis encerrando cada instancia de cada argumento en el cuerpo de una función en línea no son necesarios.

La función en línea

```
inline int square(int x) { return x * x; }
```

calcula el cuadrado de su argumento entero **x**. La llamada **square(2 + 3)** evalúa el argumento $2 + 3 = 5$ y substituye este valor dentro del cuerpo de la función. El programa de la figura 15.4 demuestra las macros **VALIDSQUARE** y **INVALIDSQUARE**, así como la función en línea **square**.

```
// Examples of valid macros, invalid macros
// and inline functions
#include <iostream.h>

#define VALIDSQUARE(x) (x) * (x)
#define INVALIDSQUARE(x) x * x

inline int square(int x) { return x * x; }

main()
{
    cout << " VALIDSQUARE(2 + 3) = "
        << VALIDSQUARE(2 + 3)
        << "\nINVALIDSQUARE(2 + 3) = "
        << INVALIDSQUARE(2 + 3)
        << "\n      square(2 + 3) = "
        << square(2 + 3) << '\n';

    return 0;
}
```

```
VALIDSQUARE(2 + 3) = 25
INVALIDSQUARE(2 + 3) = 11
square(2 + 3) = 25
```

Fig. 15.4 Macros de preprocesador y funciones en línea.

La figura 15.5 contiene una lista completa de las palabras reservadas de C++. La figura muestra las palabras reservadas comunes a C y a C++, y a continuación resalta las palabras reservadas, únicas en C++. Cada una de las palabras reservadas nuevas de C++ es explicada con detalle más adelante en el libro.

Error común de programación 15.5

C++ es un lenguaje en evolución y algunas de sus características pudieran no estar disponibles en su computadora. Usar características no puestas en práctica causará errores de sintaxis.

15.8 Parámetros por referencia

En C, todas las llamadas de función son llamadas por valor. En C las llamadas por referencia son simuladas pasando un apuntador a un objeto y obteniendo a continuación acceso al objeto desreferenciando el apuntador en la función llamada. Recuerde que en C los nombres de arreglos son ya apuntadores (constantes), por lo que los arreglos son automáticamente pasados en llamada simulada por referencia. Otros lenguaje de programación ofrecen formas directas de llamada por referencia como los parámetros **var** en Pascal. C++ corrige esta debilidad de C al ofrecer *parámetros de referencia*.

Un parámetro de referencia es un seudónimo de su argumento correspondiente. Para indicar que un parámetro de función es pasado por referencia, sólo coloque ampersand (&) después del tipo del parámetro en el prototipo de función (exactamente de la misma forma en que pondría un * después del tipo parámetro, para indicar que un parámetro es el apuntador a una variable). Utilice

Palabras reservadas en C++				
C y C++				
auto	break	case	char	const
continue	default	do	double	else
enum	extern	float	for	goto
if	int	long	register	return
short	signed	sizeof	static	struct
switch	typedef	union	unsigned	void
volatile	while			
C++ únicamente				
asm	Medio definido por la puesta en práctica de utilización de lenguaje de ensamble a lo largo de C++ (vea los manuales correspondientes a su sistema).			
catch	Maneja una excepción generada por un throw.			
class	Define una nueva clase. Pueden crearse objetos de esta clase.			
delete	Destruye un objeto de memoria creado con new.			
friend	Declara una función o una clase que sea un "friend (amigo)" de otra clase. Los amigos pueden tener acceso a todos los miembros de datos y a todas las funciones miembro de una clase.			
inline	Avisa al compilador que una función particular deberá ser generada en línea, en vez de requerir de una llamada de función.			
new	Asigna dinámicamente un objeto de memoria "en la tienda libre" —memoria adicional disponible para el programa en tiempo de ejecución. Determina automáticamente el tamaño del objeto.			
operator	Declara un operador "homónimo".			
private	Un miembro de clase accesible a funciones miembro y a funciones friend de la clase de miembros private.			
protected	Una forma extendida de acceso private; también se puede tener acceso a los miembros protected por funciones miembro de clases derivadas y amigos de clases derivadas.			
public	Un miembro de clase accesible a cualquier función.			
template	Declare como construir una clase o una función, usando una variedad de tipos.			
this	Un apuntador declarado en forma implícita en toda función de miembro no static de una clase. Señala al objeto al cual esta función miembro ha sido invocada.			
throw	Transfiere control a un manejador de excepción o termina la ejecución del programa si no puede ser localizado un manejador apropiado.			
try	Crea un bloque que contiene un conjunto de números que pudieran generar excepciones, y habilita el manejo de excepciones para cualquier excepción generada.			
virtual	Declara una función virtual.			

Fig. 15.5 Las palabras reservadas en C++.

la misma regla convencional en el encabezado de función al enlistar el tipo de parámetros. Por ejemplo, la declaración

```
int &count
```

en un encabezado de función puede ser leído "count es una referencia a int". En la llamada de función, sólo mencione la variable por su nombre y automáticamente será pasada por referencia. Entonces, el mencionar en el cuerpo de la función la variable por su nombre local, de hecho la refiere a la variable original en la función llamadora, y la variable original puede ser modificada de manera directa por la función llamada.

La figura 15.6 compara la llamada por valor, la llamada por referencia mediante apuntadores, y la llamada por referencia mediante parámetros de referencia. Los argumentos en las llamadas a squareByValue y squareByReference son idénticos. Es imposible saber si cualquiera de estas funciones modifica sus argumentos sin verificar los prototipos de función o las definiciones de función. Por esta razón, algunos programadores de C++ prefieren que los argumentos modificables sean pasados a las funciones utilizando apuntadores, y los argumentos no modificables sean pasados a las funciones utilizando referencias a constantes. Las referencias a constantes proporcionan la eficiencia del paso utilizando apuntadores y evitan la modificación de los argumentos del programa llamador. Para especificar una referencia a una constante, coloque el calificador const en la declaración de parámetros antes del especificador de tipo (en la sección 15.9 se analiza const en detalle). Note la colocación de * y de & en las listas de parámetros de ambas funciones. Algunos programadores de C++ prefieren escribir int *bPtr en vez de int* bPtr, e int& cRef en vez de int &cRef.

Práctica sana de programación 15.5

Utilice apuntadores para pasar argumentos que pudieran ser modificados por la función llamada, y utilice referencias a constantes para pasar argumentos extensos, que no serán modificados.

Error común de programación 15.6

Dado que en el cuerpo de la función llamada los parámetros de referencia se mencionan sólo por nombre, el programador pudiera de forma inadvertida tratar a los parámetros de referencia como parámetros en llamada por valor. Esto puede causar efectos colaterales inesperados, si las copias originales de las variables son modificadas por la función llamadora.

Sugerencia de rendimiento 15.2

En el caso de objetos extensos, utilice un parámetro de referencia const para simular la apariencia y la seguridad de una llamada por valor, pero evitando la sobrecarga de pasar una copia de dicho objeto extenso.

La referencia también puede ser utilizada como un seudónimo para otras variables dentro de una función. Por ejemplo, el código

```
int count = 1;      // declare integer variable count
int &c = count;      // create c as an alias for count

++c;                // increment count (using its alias)
```

incrementa la variable count utilizando su seudónimo c. Las variables de referencia deben ser inicializadas en sus declaraciones (véase las figuras 15.7 y 15.8), y no pueden ser reasignadas como seudónimos a otras variables. Una vez declarada una referencia como un seudónimo para otra variable, todas las operaciones que supuestamente se ejecuten sobre el seudónimo (es decir, sobre la referencia) de hecho se ejecutan sobre la variable original misma. El seudónimo es sólo

```
// Comparing call by value, call by reference with
// pointers, and call by reference with references
#include <iostream.h>

int squareByValue(int);
void squareByPointer(int *);
void squareByReference(int &);

main()
{
    int x = 2, y = 3, z = 4;

    cout << "x = " << x << " before squareByValue\n"
         << "Value returned by squareByValue: "
         << squareByValue(x)
         << "\nx = " << x << " after squareByValue\n\n";

    cout << "y = " << y << " before squareByPointer\n";
    squareByPointer(&y);
    cout << "y = " << y << " after squareByPointer\n\n";

    cout << "z = " << z << " before squareByReference\n";
    squareByReference(z);
    cout << "z = " << z << " after squareByReference\n";

    return 0;
}

int squareByValue(int a)
{
    return a *= a;    // caller's argument not modified
}

void squareByPointer(int *bPtr)
{
    *bPtr *= *bPtr;  // caller's argument modified
}

void squareByReference(int &cRef)
{
    cRef *= cRef;    // caller's argument modified
}
```

```
x = 2 before squareByValue
Value returned by squareByValue: 4
x = 2 after squareByValue

y = 3 before squareByPointer
y = 9 after squareByPointer

z = 4 before squareByReference
z = 16 after squareByReference
```

Fig. 15.6 Un ejemplo de llamada por referencia.

otro nombre de la variable original para el seudónimo —no se reserva espacio. Las referencias no pueden ser desreferenciadas mediante el apuntador de operador de indirección (*). Las referencias no pueden ser utilizadas para ejecutar "aritmética de referencia" (como cuando utilizamos apuntadores en aritmética de apuntadores para señalar a otros elementos de un arreglo). Otras operaciones inválidas sobre referencias incluyen señalar a referencias, tomar las direcciones de referencias, y comparar referencias (de hecho cada una de estas operaciones no generará un error de sintaxis; en vez de ello cada una de estas operaciones se efectuará en la variable para la cual la referencia es un seudónimo).

Las funciones pueden regresar apuntadores o referencias, pero esto puede resultar peligroso. Cuando se regresa un apuntador o una referencia a una variable declarada en la función llamada, la variable deberá ser declarada **static** dentro de dicha función. De lo contrario, el apuntador o referencia se refiere a una variable automática que al terminar la función se descarta; dicha variable se dice que está "indefinida" y el comportamiento de programa sería impredecible.

Error común de programación 15.7

No inicializar una variable de referencia al declararse.

Error común de programación 15.8

Intentar reasignar una referencia previa declarada como seudónimo a otra variable.

Error común de programación 15.9

Intentar desreferenciar una variable de referencia mediante un operador de indirección de apuntadores (recuerde que una referencia es un seudónimo correspondiente a una variable, y no un apuntador a una variable).

```
// References must be initialized
#include <iostream.h>

main()
{
    int x = 3, &y;    // Error: y must be initialized

    cout << "x = " << x << '\n'
         << "y = " << y << '\n';
    y = 7;
    cout << "x = " << x << '\n'
         << "y = " << y << '\n';

    return 0;
}
```

```
Compiling FIG15_7.CPP:
Error FIG15_7.CPP 6: Reference variable 'y' must be
initialized
```

Fig. 15.7 Intento de uso de una referencia no inicializada.

```
// References must be initialized
#include <iostream.h>

main()
{
    int x = 3, &y = x; // y is now an alias for x

    cout << "x = " << x << '\n'
         << "y = " << y << '\n';
    y = 7;
    cout << "x = " << x << '\n'
         << "y = " << y << '\n';

    return 0;
}
```

```
x = 3
y = 3
x = 7
y = 7
```

Fig. 15.8 Cómo utilizar una referencia inicializada.

Error común de programación 15.10

Regresar un apuntador o una referencia a una variable automática en una función llamada.

15.9 El calificador Const

En la sección 15.7 se ilustró la utilización del calificador **const** en la lista de parámetros de una función, para especificar que un argumento pasado a la función no es modificable en dicha función. El calificador **const** también puede ser utilizado para declarar las supuesta llamadas "variables constantes" (en vez de declarar constantes simbólicas en el preprocesador mediante **#define**) como en

```
const float PI = 3.14159;
```

Esta declaración genera la "variable constante" **PI** y la inicializa a **3.14159**. La variable al declararse debe ser inicializada con una expresión constante, y después no puede ser modificada (figura 15.9 y figura 15.10). Las "variables constantes" también se conocen como *constantes nombradas* o *variables de sólo lectura*. Note que el término "variable constante" es un oxímoron (contradicción), es decir, una contradicción en términos similares a "camarón jumbo" o "quemadura de congelador". (¡Por favor envíenos sus oximorones favoritos vía nuestra dirección de correo electrónico incluida en el prefacio. Gracias!).

Las variables constantes pueden ser colocadas en cualquier parte en que se espere una expresión constante. Por ejemplo, la siguiente declaración de arreglo utiliza la variable **const** para especificar el tamaño del arreglo:

```
const int arraySize = 100;
int array[arraySize];
```

```
// A const object must be initialized
main()
{
    const int x; // Error: x must be initialized

    x = 7; // Error: cannot modify a const variable

    return 0;
}
```

```
Compiling FIG15_9.CPP:
Error FIG15_9.CPP 4: Constant variable 'x' must be
    initialized
Error FIG15_9.CPP 6: Cannot modify a const object
```

Fig. 15.9 Un objeto **const** debe ser inicializado.

```
// Using a properly initialized constant variable
#include <iostream.h>

main()
{
    const int x = 7; // initialized constant variable

    cout << "The value of constant variable x is: "
         << x << '\n';

    return 0;
}
```

```
The value of constant variable x is: 7
```

Fig. 15.10 Cómo inicializar correctamente y cómo utilizar una variable constante.

Las variables constantes también pueden ser colocadas en archivos de cabecera.

Error común de programación 15.11

Usar variables **const** en declaraciones de arreglos y colocar variables **const** en archivos de cabecera (que están incluidos en varios archivos de fuente del mismo programa), ambos son ilegales en C pero legales en C++.

Práctica sana de programación 15.6

Una ventaja del uso de variables **const** en lugar de constantes simbólicas, es que las variables **const** son visibles con un depurador simbólico; las constantes simbólicas **#define** no lo son.

Existen otros usos comunes para calificador **const**. Por ejemplo, puede ser declarado un apuntador constante:

```
int *const iptr = &integer;
```

Esto declara a **iptr** como un apuntador constante a un entero. El valor al cual señala **iptr** puede ser modificado, pero **iptr** no puede ser asignado para señalar a otra posición diferente de memoria.

Un apuntador a un objeto constante puede ser declarado como sigue:

```
const int *iptr = &integer;
```

Esto declara a **iptr** como un apuntador a una constante entera. El valor al cual **iptr** se refiere no puede ser modificado mediante **iptr**, pero **iptr** puede ser asignado para señalar a otra posición de memoria. Por lo tanto, **iptr** es un apuntador a través del cual se puede leer un valor, pero éste no puede ser modificado (en efecto, un apuntador de "sólo lectura"). El compilador protege los datos referenciados por apuntadores de sólo lectura, evitando que dichos apuntadores sean asignados a apuntadores que no sean de lectura solamente.

Error común de programación 15.12

*Inicializar una variable **const** con una expresión no **const** como sería una variable que no ha sido declarada **const**.*

Error común de programación 15.13

Intentar modificar una variable constante.

Error común de programación 15.14

Intentar modificar un apuntador constante.

15.10 Asignación dinámica de memoria mediante **new** y **delete**

Los operadores **new** y **delete** de C++ le permiten a los programas llevar a cabo la asignación dinámica de memoria. En ANSI C, la asignación dinámica de memoria por lo general se lleva a cabo con las funciones estándar de biblioteca **malloc** y **free**. Considere la declaración

```
typeName *ptr;
```

donde **typeName** es cualquier tipo (como **int**, **float**, **char**, etcétera). En ANSI C, el enunciado siguiente asigna en forma dinámica un objeto **typeName**, regresa un apuntador **void** al objeto, y asigna dicho apuntador a **ptr**:

```
ptr = malloc(sizeof(typeName));
```

En C la asignación dinámica de memoria requiere una llamada de función a **malloc** y una referencia explícita al operador **sizeof** (o una mención explícita al número necesario de bytes). La memoria se asigna sobre el *montón* (memoria adicional disponible para el programa en tiempo de ejecución). También, en puestas en práctica anteriores a ANSI C, el apuntador regresado por **malloc** debe ser explícitamente convertido (cast) al tipo apropiado de apuntador con el convertidor explícito (cast) (**typeName***).

En C++, el enunciado

```
ptr = new typeName
```

asigna memoria para un objeto del tipo **typeName** partiendo de la *tienda libre* del programa (término utilizado por C++ para la memoria adicional disponible para el programa en tiempo de ejecución). El operador **new** crea automáticamente un objeto del tamaño apropiado, y regresa un apuntador del tipo apropiado. Si mediante **new** no se puede asignar memoria, se regresa un apuntador nulo (para representar un apuntador nulo los programadores C++ utilizan el valor 0, en vez de **NULL**).

Para liberar en C++ el espacio para este objeto se utiliza el siguiente enunciado:

```
delete ptr;
```

En C, se invoca la función **free** con el argumento **ptr**, a fin de desasignar memoria. El operador **delete** sólo puede ser utilizado para desasignar memoria ya asignada mediante el operador **new**. Aplicar **delete** a un apuntador previamente desasignado, puede llevar a errores inesperados durante la ejecución del programa. Aplicar **delete** a un apuntador nulo no tiene efecto en la ejecución del programa.

Error común de programación 15.15

*Desasignar memoria mediante **delete** no originalmente asignada mediante el operador **new**.*

C++ permite un *inicializador* para un objeto recién asignado. Por ejemplo,

```
float* thingPtr = new float (3.14159);
```

inicializa a 3.14159 un objeto **float** recién asignado.

También mediante **new** los arreglos pueden ser creados dinámicamente. El siguiente enunciado asigna dinámicamente un arreglo de un subíndice de 100 enteros y asigna el apuntador regresado por **new** al apuntador entero **arrayPtr**:

```
int *arrayPtr;  
arrayPtr = new int [100]; // creates array dynamically
```

Para desasignar la memoria asignada dinámicamente para **arrayPtr** por **new**, utilice el enunciado

```
delete [] arrayPtr;
```

En el capítulo 16, analizaremos la asignación dinámica de memoria de objetos **class**. Veremos que los operadores **new** y **delete** ejecutan otras tareas (como llamar de forma automática a las funciones constructor y destructor de una **class**) lo que hace que el uso de **new** y de **delete** sea más poderoso y más apropiado que el uso de **malloc** y de **free**. Por ahora, asegúrese de utilizar corchetes con **delete** al desasignar arreglos de objetos.

Error común de programación 15.16

*Usar **delete** sin corchetes ([y]) al borrar arreglos de objetos dinámicamente asignados (esto resulta un problema sólo en el caso de arreglos que contengan elementos de tipos definidos por el usuario)*

15.11 Argumentos por omisión

Las llamadas de función pudieran por lo común pasar un valor particular de un argumento. El programador puede definir que dicho argumento es un *argumento por omisión*, y el programador puede introducir el valor por omisión de dicho argumento. Cuando en una llamada de función es omitido un argumento por omisión, el valor por omisión de dicho argumento es automáticamente pasado en la llamada.

Los argumentos por omisión deben ser los argumentos que aparecen más a la derecha (los últimos) en una lista de parámetros de función. Al llamar a una función con dos o más argumentos por omisión, si un argumento omitido no es el argumento más a la derecha en la lista de argumentos, todos los argumentos a la derecha de dicho argumento también deben de ser omitidos. Los argumentos por omisión deberán ser especificados junto con la primera ocurrencia del nombre de la función —típicamente en el prototipo en un archivo de cabecera. Los argumentos por omisión también pueden ser utilizados con funciones `inline`.

En la figura 15.11 se demuestra el uso de los argumentos por omisión para calcular el volumen de una caja. A los tres argumentos se les ha dado el valor por omisión de 1. La primera llamada a la función `inline boxVolume` no especifica ningún argumento y, por lo tanto, utiliza tres valores por omisión. La segunda llamada pasa un argumento `length` y, por lo tanto, utiliza valores por omisión para los argumentos `width` y `height`. La tercera llamada pasa los argumentos para `length` y `width` y, por lo tanto, utiliza el valor por omisión para el argumento `height`. La última llamada pasa argumentos para `length`, `width` y `height` y, por lo tanto, no utiliza valores por omisión.

Práctica sana de programación 15.7

El uso de argumentos por omisión puede simplificar la escritura de las llamadas de función. Sin embargo, algunos programadores sienten que resulta más claro especificar explícitamente todos los argumentos.

Error común de programación 15.17

Especificar e intentar utilizar un argumento por omisión que no es el argumento más a la derecha (el último) (simultáneamente dar como por omisión todos los argumentos más a la derecha).

15.12 Operador de resolución de alcance unario

Es posible declarar variables locales y globales con un mismo nombre. En C, en tanto esté en alcance la variable local, todas las referencias a dicho nombre de variable pertenecerán a la variable local —dentro del alcance la variable local la variable global no estará visible. C++ dispone del *operador de resolución de alcance unario* (`::`) para tener acceso a una variable global cuando está en alcance una variable local con el mismo nombre. El operador de resolución de alcance unario no puede ser utilizado para tener acceso a una variable del mismo nombre en un bloque externo. Se puede tener acceso a una variable global directa, sin el operador de resolución de alcance unario, si el nombre de la variable global no es el mismo que el nombre de la variable local en alcance. En el capítulo 16, analizaremos el uso del *operador de resolución de alcance binario* con clases.

En la figura 15.12 se demuestra el operador de resolución de alcance unario con variables locales y globales del mismo nombre. A fin de enfatizar que las versiones local y global de la variable `value` son distintas, el programa declara una de las variables como `float` y la otra como `int`.

```
// Using default arguments
#include <iostream.h>

// Calculate the volume of a box
inline int boxVolume(int length = 1, int width = 1,
                    int height = 1)
{ return length * width * height; }

main()
{
    cout << "The default box volume is: "
          << boxVolume()
          << "\n\nThe volume of a box with length 10,\n"
          << "width 1 and height 1 is: "
          << boxVolume(10)
          << "\n\nThe volume of a box with length 10,\n"
          << "width 5 and height 1 is: "
          << boxVolume(10, 5)
          << "\n\nThe volume of a box with length 10,\n"
          << "width 5 and height 2 is: "
          << boxVolume(10, 5, 2)
          << '\n';

    return 0;
}
```

```
The default box volume is: 1

The volume of a box with length 10,
width 1 and height 1 is: 10

The volume of a box with length 10,
width 5 and height 1 is: 50

The volume of a box with length 10,
width 5 and height 2 is: 100
```

Fig. 15.11 Cómo utilizar los argumentos por omisión.

Error común de programación 15.18

Intentar tener acceso a una variable no global en un bloque externo, utilizando un operador de resolución de alcance unario.

Práctica sana de programación 15.8

Evite usar dentro de un programa variables del mismo nombre para distintos fines. Aunque en varias circunstancias esto es permitido, puede resultar confuso.

15.13 Homonimia de funciones

En C, declarar dos funciones del mismo nombre en el mismo programa es un error de sintaxis. C++ permite que sean definidas varias funciones del mismo nombre, siempre que estos nombres

```
// Using the unary scope resolution operator
#include <iostream.h>

float value = 1.2345;

main()
{
    int value = 7;

    cout << "Local value = " << value
          << "\nGlobal value = " << ::value << '\n';

    return 0;
}
```

```
Local value = 7
Global value = 1.2345
```

Fig. 15.12 Cómo utilizar el operador de resolución de alcance unario.

de funciones indiquen diferentes conjuntos de parámetros (por lo menos en lo que se refiere a sus tipos). Esta capacidad se llama *homonimia de funciones*. Cuando se llama a una función homónima, el compilador C++ selecciona de forma automática la función correcta examinando el número, tipos y orden de los argumentos de la llamada. La homonimia de la función se utiliza por lo común para crear varias funciones del mismo nombre, que ejecutan tareas similares, sobre tipos de datos diferentes.

Práctica sana de programación 15.9

La homonimia de funciones que ejecutan tareas muy relacionadas, puede hacer que los programas sean más legibles y comprensibles.

La figura 15.13 utiliza la función homónima **square** para calcular el cuadrado de un **int** y el cuadrado de un **double**. En el capítulo 17 analizaremos cómo hacer la homonimia de operadores para definir cómo deberán de operar sobre objetos de tipos de datos definidos por el usuario. En la sección 15.15 se presentan las funciones plantilla para la ejecución de tareas idénticas sobre muchos distintos tipos de datos. En el capítulo 20 se analizan las funciones plantilla y las clases plantilla con detalle.

Se pueden distinguir las funciones homónimas mediante su *firma* —una combinación del nombre de la función y de sus tipos de parámetros. El compilador codifica cada identificador de función en forma especial (conocido a veces como *mutilación de nombre o decoración de nombre*), utilizando el número y el tipo de sus parámetros, a fin de habilitar un *enlace a prueba de tipo*. Un enlace a prueba de tipo asegura que se llama a la función homónima correcta, y que los argumentos concuerdan con los parámetros. El compilador detecta y reporta los errores de enlace. El programa de la figura 15.14 fue compilado por el compilador Borland C++. Los nombres de función cifrados, producidos en lenguaje de ensamble por Borland C++, aparecen en la ventana de salida. Cada nombre “decorado” empieza con un **@**, seguido por el nombre de la función. La lista de parámetros codificados empieza con **\$q**. En la lista de parámetros correspondiente a la función **nothing2**, **z c** representa a un **char**, **i** representa a un **int**, **pf**

```
// Using overloaded functions
#include <iostream.h>

int square(int x) { return x * x; }

double square(double y) { return y * y; }

main()
{
    cout << "The square of integer 7 is "
          << square(7)
          << "\nThe square of double 7.5 is "
          << square(7.5) << '\n';

    return 0;
}
```

```
The square of integer 7 is 49
The square of double 7.5 is 56.25
```

Fig. 15.13 Cómo utilizar funciones homónimas.

representa a un **float ***, y **pd** representa a un **double ***. En la lista de parámetros correspondiente a la función **nothing1**, **i** representa a un **int**, **f** representa a un **float**, **zc** representa a un **char**, y **pi** representa a un **int***. Las dos funciones **square** se reconocen por medio de sus listas de parámetros; una especifica **d** para **double** y la otra especifica **i** para **int**. Los tipos de regreso de las cuatro funciones no se especifican en los nombres codificados. Note que el codificado de los nombres de funciones es específico a cada compilador. Por lo tanto, una función compilada con Borland C++ pudiera no tener el mismo nombre “decorado” que en otros compiladores.

Las funciones homónimas pueden tener diferentes tipos de regreso, pero deben tener diferentes listas de parámetros.

Error común de programación 15.19

Generará un error de sintaxis crear funciones homónimas con listas de parámetros idénticas pero distintos tipos de regreso.

Para distinguir entre funciones con un mismo nombre el compilador sólo utiliza las listas de parámetros. Las funciones homónimas no necesariamente tienen el mismo número de parámetros. Al hacer la homonimia de funciones utilizando parámetros por omisión, los programadores deberán tener cuidado, ya que ello podría causar ambigüedad.

Error común de programación 15.20

Una función con argumentos por omisión omitidos pudiera ser llamada en forma idéntica a otra función homónima; esto resultará en un error de sintaxis.

Error común de programación 15.21

*La homonimia ayuda a eliminar el uso de las macros **#define** de C, que ejecutan tareas sobre múltiples tipos de datos, y utiliza las características poderosas de verificación de tipo de C++, que al utilizar macros no están disponibles.*

```
// Name mangling
int square(int x) { return x * x; }

double square(double y) { return y * y; }

void nothing1(int a, float b, char c, int *d) {}

char *nothing2(char a, int b, float *c, double *d) { return 0; }

main()
{
    return 0;
}
```

```
public _main
public @nothing2$zqipfpd
public @nothing1$zqipfpd
public @square$qd
public @square$qi
```

Fig. 15.14 Decoración de nombres para habilitar enlaces a prueba de tipo.

15.14 Especificaciones de enlace

Es posible, desde un programa C++, llamar funciones escritas y compiladas con un compilador C. Como se indicó en la sección 15.13, C++ cifra en especial los nombres de la función para enlaces a prueba de tipo. C, por su parte, no codifica sus nombres de función. Por lo tanto, cuando se haga algún intento para enlazar el código C con código C++, una función compilada en C no será reconocida, porque el código C++ espera un nombre de función especialmente codificado. C++ le permite al programador dar *especificaciones de enlace* para informar al compilador que una función fue compilada en un compilador C, y evitar que el nombre de la función sea codificado por dicho compilador C++. Las especificaciones de enlace son útiles cuando se han desarrollado extensas bibliotecas de funciones especializadas, y el usuario, o no tiene acceso al código fuente para recompilarlas en C++, o no tiene el tiempo para convertir las funciones de bibliotecas de C a C++.

Para informar al compilador que una o varias funciones han sido compiladas en C, escriba los prototipos de función como sigue:

```
extern "C" function prototype // single function

extern "C" // multiple functions
{
    function prototypes
}
```

Estas declaraciones le informan al compilador que las funciones especificadas no están compiladas en C++, por lo que no deberá hacerse sobre las funciones enlistadas en la especificación de enlace el codificado de nombres. Estas funciones entonces podrán ser correctamente enlazadas

con el programa. Los entornos C++ incluyen normalmente las bibliotecas estándar de C y para dichas funciones no se requiere que el programador utilice especificaciones de enlace.

15.15 Plantillas de función

Las funciones homónimas se utilizan por lo regular para ejecutar operaciones similares sobre distintos tipos de datos. Si para cada tipo las operaciones son idénticas, esto se puede llevar a cabo de manera más compacta mediante *plantillas de función*, una capacidad introducida en versiones recientes de C++. El programador escribe una definición de plantilla de función. Basado en los tipos de los argumentos proporcionados en las llamadas a esta función, C++ genera de forma automática funciones de código objeto por separado para manejar cada tipo de llamada en forma correcta. En C, esta tarea puede ser llevada a cabo mediante macros creados con **#define**. Sin embargo, las macros presentan la posibilidad de serios efectos colaterales y no permiten que el compilador ejecute verificación de tipo. Las plantillas de función proporcionan, como las macros, una solución compacta, pero permiten verificación completa de tipo.

Todas las definiciones de plantillas de función empiezan con la palabra reservada **template**, seguida por una lista de parámetros formales a la plantilla de función encerrados en corchetes angulares (< y >). Cada parámetro formal es precedido por la palabra reservada **class**. Los parámetros formales se utilizan como tipos incorporados, o como tipos definidos por el usuario, para definir los tipos de los argumentos a la función, para definir el tipo de regreso de la función, y para declarar variables dentro de la función. A continuación se coloca la definición de función y se define como cualquier otra función.

La siguiente plantilla de función también es utilizada en el programa completo de la figura 15.15.

```
template <CLASS T>
void printArray(T *array, const int count)
{
    for (int i = 0; i < count; i++)
        cout << array[i] << " ";
    cout << '\n';
}
```

Esta plantilla de función declara un parámetro formal único **T** como el tipo del arreglo a imprimirse mediante la función **printArray**. Cuando el compilador detecta la llamada a **printArray** en el código fuente del programa, se substituye el tipo del primer argumento de **printArray** por la **T** en toda la definición de la plantilla, y C++ genera una plantilla de función completa, para la impresión de un arreglo de un tipo de datos especificado. A continuación se compila la nueva función creada. En la figura 15.15 se ejemplifican tres funciones una espera un arreglo **int**, otra espera un arreglo **float** y otra un arreglo **char**. La ejemplificación del tipo **int** es:

```
void printArray(int *array, const int count)
{
    for (int i = 0; i < count; i++)
        cout << array[i] << " ";
    cout << '\n';
}
```

Cada parámetro formal en la definición de plantilla debe aparecer en la lista de parámetros de la función por lo menos una vez. El nombre de un parámetro formal sólo puede ser utilizado

una vez en la lista de parámetros formales de la definición de plantilla. Los nombres de los parámetros formales entre funciones de plantilla no es necesario que deban ser únicos.

En la figura 15.15 se ilustra el uso de la función de plantilla `printArray`.

Error común de programación 15.22

No colocar la palabra reservada `class` antes de cualquier parámetro formal de una plantilla de función.

Error común de programación 15.23

No usar en la firma de función todos los parámetros formales de una plantilla de función.

```
// Using template functions
#include <iostream.h>

template <class T>
void printArray(T *array, const int count)
{
    for (int i = 0; i < count; i++)
        cout << array[i] << " ";

    cout << '\n';
}

main()
{
    const int aCount = 5, bCount = 7, cCount = 6;
    int a[aCount] = {1, 2, 3, 4, 5};
    float b[bCount] = {1.1, 2.2, 3.3, 4.4, 5.5, 6.6, 7.7};
    char c[cCount] = "HELLO"; // 6th position for null

    cout << "Array a contains:\n";
    printArray(a, aCount); // integer version

    cout << "Array b contains:\n";
    printArray(b, bCount); // float version

    cout << "Array c contains:\n";
    printArray(c, cCount); // character version

    return 0;
}
```

```
Array a contains:
1 2 3 4 5
Array b contains:
1.1 2.2 3.3 4.4 5.5 6.6 7.7
Array c contains:
H E L L O
```

Fig. 15.15 Cómo utilizar funciones plantilla.

Resumen

- C++ es un superconjunto de C, por lo que los programadores pueden utilizar un compilador C++ para compilar sus programas existentes en C, y de ahí modificar de forma gradual estos programas a C++.
- C requiere que un comentario quede delimitado por `/*` y `*/`. C++ permite que un comentario empiece con `//` y que termine al final de la línea.
- C++ permite que las declaraciones aparezcan en cualquier parte en que un enunciado ejecutable pueda aparecer. Las declaraciones variables pueden también aparecer en la sección de inicialización de una estructura `for`.
- C++ proporciona una alternativa a las funciones `printf` y `scanf` para manejar la entrada/salida de los tipos de datos estándar y de las cadenas. El flujo de salida `cout`, y el operador `<<` (operador de inserción de flujo, que se lee "colocar en") le permite la salida de los datos. El flujo de entradas `cin` y el operador `>>` (el operador de extracción de flujo, que se lee "obtener de") permite que se introduzcan datos.
- C++ le permite al programador que cree tipos de datos definidos por el usuario, mediante las palabras reservadas `enum`, `struct`, `union` y `class`. El nombre de etiqueta de la enumeración, estructura, unión o clase puede entonces ser utilizada para definir variables del nuevo tipo.
- Los programas no se compilan a menos de que un prototipo de función esté incluida para cada función o una función esté definida antes de su uso (en cuyo caso no es requerida un prototipo de función por separado).
- Una función que no regrese un valor se declara con el tipo de regreso `void`. Si se hace algún intento ya sea de que la función regrese un valor o de utilizar en la función llamadora el resultado de la invocación de la función, el compilador generará un error.
- En C++, una lista de parámetros vacía se especifica con paréntesis vacíos, o con `void` entre paréntesis. En C, una lista de parámetros vacía desactiva la verificación de argumentos para la función.
- Las funciones en línea eliminan la sobrecarga de llamadas de función. El programador utiliza la palabra reservada `inline` para avisarle al compilador que genere código de función en línea (siempre que sea posible) a fin de minimizar llamadas de función. El compilador puede decidir ignorar el consejo `inline`.
- C++ ofrece una forma directa de llamada por referencia mediante el uso de parámetros de referencia. Para indicar que un parámetro de función es pasado por referencia, haga seguir al tipo de parámetro en el prototipo de función por un `&`. En la llamada de función, mencione la variable por nombre y será pasada en llamada por referencia. En la función llamada, el mencionar la variable por su nombre local, de hecho se refiere a la variable original en la función llamadora. Por lo tanto, la variable original puede ser modificada directamente por la función llamada.
- Los parámetros de referencia también pueden ser creados para uso local, como seudónimo de otras variables dentro de una función. Las variables de referencia deben ser inicializadas en sus declaraciones, y no pueden ser reasignadas como seudónimos de otras variables. Una vez declarada una variable de referencia como un seudónimo de otra variable, todas las operaciones supuestamente ejecutadas sobre el seudónimo de hecho se ejecutan sobre la variable.

- El calificador **const** también puede crear "variables constantes". Una variable constante debe ser inicializada al declarar la variable con una expresión constante, y después no puede ser modificada. Las variables constantes a menudo se llaman constantes o variables de sólo lectura. Las variables constantes pueden ser colocadas en cualquier lugar en donde se espere una expresión constante. Otros usos comunes del calificador **const** incluye apuntadores constantes, apuntadores a constantes y referencias constantes.
- Los operadores **new** y **delete** en C++ ofrecen una mejor forma de llevar a cabo la asignación dinámica de memoria que con las llamadas de función **malloc** y **free** en C. El operador **new** toma como operando un tipo, crea automáticamente un objeto del tamaño correcto, y regresa un apuntador del tipo correcto. El operador **delete** toma un apuntador a un objeto asignado con **new** y libera la memoria.
- C++ permite al programador especificar los argumentos por omisión y sus valores por omisión. Si en una llamada a una función se omite un argumento por omisión, se utiliza el valor por omisión de dicho argumento. Los argumentos por omisión deben ser los argumentos más a la derecha (los últimos) en la lista de parámetros de una función. Los argumentos por omisión deberán ser especificados en la primera ocurrencia del nombre de función.
- El *operador de resolución de alcance unario* (**::**) le permite a un programa tener acceso a una variable global, cuando está en alcance una variable local con el mismo nombre.
- Es posible definir varias funciones con el mismo nombre, pero con distintos tipos de parámetros. Esto se llama homonimia de función. Cuando es llamada una función homónima, el compilador selecciona automáticamente la función correcta, mediante el examen del número y tipo de los argumentos en la llamada.
- Las funciones homónimas pueden tener diferentes valores de regreso, y deben de tener diferentes listas de parámetros. Dos funciones que sólo difieran en el tipo de regreso darán como resultado un error de compilación.
- En algunos casos, es necesario llamar funciones escritas y compiladas con un compilador de C (por ejemplo, las funciones de biblioteca especializadas de una empresa que no hayan sido convertidas a C++ y recompiladas). C++ procesa los nombres de sus funciones en forma distinta a como lo hace C. Por lo tanto cuando se intente enlazar código C con código C++, una función compilada en C no será reconocida. C++ dispone de especificaciones de enlace, para informar al compilador que una función fue compilada sobre un compilador C, y evitar que se procese el nombre de la función como si fuera una función C++.
- Las plantillas de función permiten la creación de funciones que ejecutan las mismas operaciones sobre distintos tipos de datos, pero la plantilla de función se define sólo una vez.

Terminología

sufijo ampersand (&)	operador delete
asm	objetos dinámicos
catch	tienda libre
cin (flujo de entrada)	friend
const	homonimia de funciones
variable constante	operador "obtener de" (>>)
cout (flujo de salida)	inicializador
class	función miembro inline
argumentos de función por omisión	<iostream.h>

biblioteca **iostream**
 especificaciones de enlace
 constante nombrada
 operador **new**
 palabra reservada **operator**
 homonimia
 operador "colocar en" (<<)
 variable de lectura solamente
 parámetro de referencia
 tipos de referencia
 signatura

comentario en una sola línea (//)
 operador de extracción de flujo (>>)
 operador de inserción de flujo (<<)
template
 función plantilla
this
throw
try
 enlace a prueba de tipo
 operador de resolución de alcance unario (**::**)
virtual

Errores comunes de programación

- 15.1 Olvidar cerrar un comentario de estilo C mediante ***/**.
- 15.2 Declarar una variable después de que haya sido referenciada en un enunciado.
- 15.3 Un intento de regresar un valor de una función **void** o de utilizar el resultado de una llamada a una función **void**.
- 15.4 Los programas en C++ no se compilarán a menos de que sean proporcionadas los prototipos de función para cada una de las funciones, o que cada función quede definida antes de ser utilizada.
- 15.5 C++ es un lenguaje en evolución y algunas de sus características pudieran no estar disponibles en su computadora. Usar características no puestas en práctica causará errores de sintaxis.
- 15.6 Dado que en el cuerpo de la función llamada los parámetros de referencia se mencionan sólo por nombre, el programador pudiera pasar por inadvertido y tratar a los parámetros de referencia como parámetros en llamada por valor. Esto puede causar efectos colaterales inesperados, si las copias originales de las variables son modificadas por la función llamadora.
- 15.7 No inicializar una variable de referencia al declararse.
- 15.8 Intentar reasignar una referencia ya declarada como seudónimo a otra variable.
- 15.9 Intentar desreferenciar una variable de referencia mediante un operador de indirección de apuntadores (recuerde que una referencia es un seudónimo correspondiente a una variable, y no un apuntador a una variable).
- 15.10 Regresar un apuntador o una referencia a una variable automática en una función llamada.
- 15.11 Usar variables **const** en declaraciones de arreglos y colocar variables **const** en archivos de cabecera (que están incluidos en varios archivos de fuente del mismo programa), ambos son ilegales en C pero legales en C++.
- 15.12 Inicializar una variable **const** con una expresión no **const** como sería una variable que no ha sido declarada **const**.
- 15.13 Intentar modificar una variable constante.
- 15.14 Intentar modificar un apuntador constante.
- 15.15 Desasignar memoria mediante **delete** no originalmente asignada mediante el operador **new**.
- 15.16 Usar **delete** sin corchetes (**[** y **]**) al borrar arreglos de objetos dinámicamente asignados (esto resulta un problema sólo en el caso de arreglos que contengan elementos de tipos definidos por el usuario)
- 15.17 Especificar e intentar utilizar un argumento por omisión que no es el argumento más a la derecha (el último) (simultáneamente dar como por omisión todos los argumentos más a la derecha).
- 15.18 Intentar tener acceso a una variable no global en un bloque externo, utilizando un operador de resolución de alcance unario.
- 15.19 Generará un error de sintaxis crear funciones homónimas con listas de parámetros idénticas pero distintos tipos de regreso.
- 15.20 Una función con argumentos por omisión omitidos pudiera ser llamada en forma idéntica a otra función homónima; esto resultará en un error de sintaxis.

- 15.21 La homonimia ayuda a eliminar el uso de las macros `#define` de C, que ejecutan tareas sobre múltiples tipos de datos, y utiliza las características poderosas de verificación de tipo de C++, que al utilizar macros no están disponibles.
- 15.22 No colocar la palabra reservada `class` antes de cualquier parámetro formal de una plantilla de función.
- 15.23 No usar en la firma de función todos los parámetros formales de una plantilla de función.

Prácticas sanas de programación

- 15.1 Usar los comentarios `//` de estilo C++, evita errores de comentarios sin terminar, que ocurren al olvidarse de cerrar los comentarios de estilo C con `*/`.
- 15.2 Utilizar entrada/salida orientada a flujo de tipo C++ hace los programas más legibles (y menos sujetos a errores), que sus contrapartidas escritas en C mediante las llamadas de función `printf` y `scanf`.
- 15.3 Colocar la declaración de una variable cerca de su primer uso puede hacer los programas más legibles.
- 15.4 El calificador `inline` deberá ser utilizado únicamente tratándose de funciones pequeñas, de uso frecuente.
- 15.5 Utilice apuntadores para pasar argumentos que pudieran ser modificados por la función llamada, y utilice referencias a constantes para pasar argumentos extensos, que no serán modificados.
- 15.6 Una ventaja del uso de variables `const` en lugar de constantes simbólicas, es que las variables `const` son visibles con un depurador simbólico; las constantes simbólicas `#define` no lo son.
- 15.7 El uso de argumentos por omisión puede simplificar la escritura de las llamadas de función. Sin embargo, algunos programadores sienten que resulta más claro especificar de manera explícita todos los argumentos.
- 15.8 Evite usar dentro de un programa variables del mismo nombre para distintos fines. Aunque en varias circunstancias esto es permitido, puede resultar confuso.
- 15.9 La homonimia de funciones que ejecutan tareas muy relacionadas, puede hacer que los programas sean más legibles y comprensibles.

Sugerencias de rendimiento

- 15.1 Usar funciones `inline` puede reducir tiempo de ejecución, pero puede aumentar el tamaño del programa.
- 15.2 En el caso de objetos extensos, utilice un parámetro de referencia `const` para simular la apariencia y la seguridad de una llamada por valor, pero evitando la sobrecarga de pasar una copia de dicho objeto extenso.

Sugerencia de portabilidad

- 15.1 El significado de una lista de función de parámetros vacía es dramáticamente distinto en C++ que en C. En C, significa que se deshabilita toda verificación de argumentos. En C++, significa que la función no toma argumentos. Por lo tanto, si se compilan en C++, los programas en C que utilizan esta característica pudieran ejecutarse en forma diferente.

Observación de ingeniería de software

- 15.1 Cualquier modificación a una función `inline` requiere que sean recompilados todos los clientes de dicha función. Esto pudiera resultar de importancia en algunas situaciones de desarrollo y mantenimiento de programas.

Ejercicios de autoevaluación

- 15.1 Llene cada uno de los siguientes espacios vacíos:
- En C++, es posible tener varias funciones con el mismo nombre, cada una de ellas operando sobre diferentes tipos o números de argumentos. Esto se llama _____ de funciones.
 - La _____ permite el acceso a una global variable con el mismo nombre que una variable en el alcance actual.
 - El operador _____ asigna dinámicamente un nuevo objeto.
 - En C, suponga que `a` y `b` son variables enteras y que formamos la suma `a + b`. Ahora suponga que `c` y `d` son variables de punto flotante y que formamos la suma `c + d`. Los dos operadores `+` se están utilizando aquí para fines claramente distintos. Esto es un ejemplo de una propiedad que tiene C++, y que también tiene C, llamada _____ de función.
 - Dos objetos de flujo que hemos analizado y que están predefinidos en C++ son _____ y _____.
 - Las palabras reservadas _____, _____, _____ y _____ son utilizadas para crear nuevos tipos de datos en C++.
 - El calificador _____ sólo se utiliza para declarar variables de lectura.
 - C++ ofrece _____ para permitir a las funciones compiladas en un compilador de C que se enlacen correctamente con un programa de C++.
 - Las funciones _____ habilitan a un sola función para que pueda ser definida para ejecutar una tarea sobre muchos tipos distintos de datos.
- 15.2 (Verdadero/falso). Al escribir un comentario en forma de bloque que requiera de muchas líneas de texto, es más conciso el utilizar el delimitador de comentarios de C++ `//` que los delimitadores normales de C `/*` y `*/`.
- 15.3 En C++, ¿por qué debería un prototipo de función contener una declaración de tipo de parámetro como `float&`?
- 15.4 (Verdadero/falso). Todas las llamadas en C++ se efectúan en llamada por valor.
- 15.5 Explique por qué en C++ los operadores `<<` y `>>` son de homonimia.
- 15.6 Escriba segmentos de programa en C++ para que lleven a cabo cada uno de los siguientes:
- Escriba la cadena `"Welcome to C!"` al flujo de salida estándar `cout`.
 - Lea el valor de la variable `age` a partir del flujo de entrada estándar `cin`.
- 15.7 Escriba un programa en C++ completo que utilice entrada/salida de flujo y una función `inline` llamada `sphereVolume` para solicitarle al usuario el radio de una esfera, y calcular e imprimir el volumen de dicha esfera, utilizando la asignación `volume = (4/3) PI * pow(radius, 3)`.
- 15.8 ¿Qué pasaría en C si declarase la misma función dos veces con distintos tipos de argumentos? ¿Qué pasaría en C++?

Respuestas a los ejercicios de autoevaluación

- 15.1 a) homonimia. b) operador de resolución de alcance unario (`::`). c) `new`. d) homonimia. e) `cin`, `cout`. f) `enum`, `struct`, `union`, `class`. g) `const`. h) especificaciones de enlace. i) plantilla.
- 15.2 Falso. Los delimitadores de tipo C normales, necesita cada uno sólo ser escrito una vez, en tanto que el delimitador C++ `//` necesita aparecer al principio de cada nueva línea.
- 15.3 Dado que el programador está declarando un parámetro de referencia del tipo "referencia a" `float` para tener acceso mediante llamada por referencia a la variable de argumento original.
- 15.4 Falso. C++ permite llamada por referencia directa mediante el uso de parámetros de referencia en adición al uso de apuntadores.

15.5 El operador >> es a la vez el operador de desplazamiento a la derecha y el operador de extracción de flujo, dependiendo de dónde está utilizado. El operador << es a la vez el operador de desplazamiento de la izquierda y el operador de inserción de flujo dependiendo de dónde se utilice.

15.6 a) `cout << "Welcome to C!";`
b) `cin >> age;`

15.7 `// Inline function that calculates the volume of a sphere`
`"include <iostream.h>`

`const float PI = 3.24159;`

`inline float sphereVolume(const float r) {return`
`4.0 / 3.0 * PI * r * r * r;}`

`main()`

`{`

`float radius;`

`cout << "Enter the length of the radius of your sphere: ";`

`cin >> radius;`

`cout << "Volume of sphere with radius " << radius <<`

`"is " << sphereVolume(radius) << '\n';`

`return 0;`

`}`

15.8 En C obtendría un error de compilación indicando que las dos funciones tienen el mismo nombre. En C++ esto es un ejemplo de homonimia de funciones lo que está permitido, y no habría error.

Ejercicios

15.9 Suponga una organización que actualmente enfatiza la programación en C, y desea convertir a un entorno de programación orientado a objetos en C++. ¿Cuál sería la estrategia apropiada para pasar de entornos de C a entornos de C++?

15.10 Escriba enunciados orientados a flujo de tipo C++ para llevar a cabo cada uno de los siguientes:

a) Mostrar "HELLO" en la pantalla.

b) Introducir un valor para la variable `float` llamada `temperature` desde el teclado.

15.11 Compare la entrada/salida de tipo `printf/scanf` con la entrada/salida orientada a flujos de tipo C++.

15.12 En este capítulo, mencionamos que existen muchas áreas en la cual C++ "sabe" automáticamente qué tipos utilizar. Enliste tantas de éstas como usted pueda.

15.13 Escriba un programa C++ que utilice entradas/salidas orientadas a flujos, que solicite siete enteros y determine e imprima su máximo.

15.14 Escriba un programa C++ completo que utilice entrada/salida de flujos y una función `inline` de nombre `circleArea`, para solicitarle al usuario el radio de un círculo, y que calcule e imprima el área de dicho círculo.

15.15 Escriba un programa C++ completo con las tres funciones alternas especificadas más abajo, que cada una de ellas sólo añada 1 a la variable `count` definida en `main`. A continuación compare los tres métodos alternos. Las tres funciones son

a) Función `add1CallByValue`, que pasa una copia de `count` en llamada por valor, añade 1 a la copia y regresa el nuevo valor.

b) Función `add1ByPointer`, que pasa el acceso a la variable `count` vía una llamada por referencia simulada mediante apuntadores, y utiliza el operador de desreferenciación `*` para añadir uno a la copia original de `count` en `main`.

c) La función `add1ByReference`, que pasa `count` con una llamada por referencia verdadera vía un parámetro de referencia, y añade 1 a la copia original de `count` mediante su seudónimo (es decir el parámetro de referencia).

15.16 ¿Cuál es el propósito del operador de resolución de alcance unario?

15.17 Compare la asignación dinámica de memoria mediante los operadores de C++ `new` y `delete`, con asignación de memoria dinámica mediante las funciones de la biblioteca estándar de C `malloc` y `free`.

15.18 Enliste las varias características de C++ que fueron presentadas en este capítulo, y que representan el conjunto de mejoras no orientadas a objetos a C.

15.19 Escriba un programa que utilice una plantilla de función llamada `min`, para determinar el menor de dos argumentos. Pruebe el programa utilizando pares de enteros de caracteres y de números de punto flotante.

15.20 Escriba un programa que utilice una plantilla de función llamada `max`, para determinar el mayor de tres argumentos. Pruebe el programa utilizando pares de enteros, caracteres y números de punto flotante.

15.21 Determine si los siguientes segmentos de programa contienen errores. Para cada uno de los errores, explique cómo deben ser corregidos. Nota: para un segmento particular del programa, es posible que dentro de dicho segmento no existan errores.

a) `main()`

`{`

`cout << x;`

`int x = 7;`

`return 0;`

`}`

b) `template <class A>`

`int sum(int num1, int num2, int num3)`

`{`

`return num1 + num2 + num3;`

`}`

c) `/* This is a comment`

`d) void printResults(int x, int y)`

`{`

`cout << "The sum is " << x + y << '\n';`

`return x + y;`

`}`

e) `char *s = "character string";`

`delete s;`

f) `float &ref;`

`*ref = 7;`

g) `int x;`

`const int y = x;`

h) `template <A>`

`A product(A num1, A num2, A num3)`

`{`

`return num1 * num2 * num3;`

`}`

i) `const double pi = 3.14159;`

`pi = 3;`

j) `// This is a comment`

k) `char *s = new char[10];`

`delete s;`

l) `double cube(int);`

`int cube(int);`

16

Clases y abstracción de datos

Objetivos

- Comprender los conceptos de ingeniería de software correspondientes a encapsulado y ocultamiento de datos.
- Comprender las nociones de abstracción de datos y de tipos de datos abstractos (ADT).
- Ser capaz de crear ADT, es decir clases, en C++.
- Comprender como crear, utilizar y destruir objetos de clase.
- Ser capaz de controlar el acceso a miembros de objetos de datos y a funciones miembro.
- Empezar a valorizar las ventajas de la orientación a objetos.

Mi objeto todo sublime, terminaré a tiempo.

W. S. Gilbert

¿Es éste un mundo para ocultar virtudes?

William Shakespeare

Décimo segunda noche

Tienes bien merecidos a tus sirvientes públicos.

Adlai Stevenson

Caras íntimas en lugares públicos

son más sabias y más agradables

Que caras públicas en lugares íntimos.

W. H. Auden